# Disk-Resident Vector Similarity Search: A Survey

Yitong Song, Huiling Li, Zheng Wu, Lanjing Yi, Bojian Zhu, Xuanhe Zhou, Xin Huang, and Jianliang Xu, *Fellow, IEEE*

*(Survey Paper)*

***Abstract*—Vector similarity search (VSS) has become ubiquitous in modern multimodal data retrieval systems, supporting a wide range of applications such as document retrieval, music recognition, image search, and code assistants. With the rapid growth of vector datasets and the prohibitive cost of main memory, VSS techniques are increasingly shifting from memory-resident to disk-resident settings. Unlike memory-resident designs, which keep both raw vectors and index structures entirely in memory, disk-resident VSS places raw vectors and fine-grained index structures on secondary storage while retaining only lightweight components in memory. Therefore, disk-resident VSS must explicitly account for data access costs (i.e., I/O) and block-level layouts, leading to index organizations and query execution strategies that are fundamentally different from those used in memory-resident systems.**

**This survey provides the first systematic review of disk-resident VSS, a critical yet challenging task for modern large-scale data systems. We introduce a unified framework that categorizes existing techniques into IVF-based, graph-based, and tree-based paradigms, based on the primary index structure used for filtering. For each category, we further decompose the overall design into key technical components, including index construction, block-aware layouts, query execution strategies, and update mechanisms. Moreover, we summarize commonly used datasets to facilitate reproducible benchmarking, and identify open challenges and promising directions for future research**

***Index Terms*—High-dimensional vector, approximate nearest neighbor search, disk-resident search.**

## I. Introduction

VECTOR Similarity Search (VSS) retrieves the vectors most similar to a given query from large-scale, high-dimensional datasets. It has become a fundamental building block of modern data-intensive applications, including information retrieval [1], music recognition [2], image search [3], and code assistants [4]. More. recently, VSS has also emerged as a core component of retrieval-augmented generation (RAG) for large language models (LLMs) [5], [6], [7], where vector databases store billions of embeddings and VSS is used to retrieve semantically relevant knowledge to enhance factuality and mitigate hallucinations during LLM generation.

Early VSS solutions are predominantly memory-resident, assuming that both high-dimensional vectors and their index structures can be fully stored in main memory. Under this assumption, a broad spectrum of in-memory index structures has been developed to accelerate VSS [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]. However, as vector datasets continue to grow and memory remains both limited and expensive, maintaining large collections of vectors together with memory-intensive indexes entirely in memory becomes increasingly impractical [18], [19], [20]. This challenge has motivated a new search paradigm, namely *Disk-Resident VSS*, in which vectors and fine-grained index components are placed on secondary storage (typically SSDs), while only lightweight structures and cached data are retained in memory to guide retrieval.

Different from purely memory-resident approaches, disk-resident VSS methods must explicitly consider hybrid memory–disk storage architectures and block (i.e., page) layouts, as well as the dominant I/O costs that are negligible in memory-only settings. To address these challenges, recent studies have proposed a wide range of techniques, such as balanced data partitioning [19], [21], block-aware index construction [22], [23], [24], [25], locality-aware disk layout [20], [26], [27], and hybrid caching strategies [24], [26], [28]. However, these methods differ substantially in their underlying index paradigms, physical layouts, query execution strategies, and update mechanisms, resulting in a highly heterogeneous design space that is difficult to navigate. This fragmentation motivates the need for a systematic survey that provides a unified view of disk-resident VSS and addresses the following key questions: (1) What types of disk-resident VSS indexes exist, and which application scenarios are they best suited for? (2) How does each category construct its index structures and organize them along with raw vectors across heterogeneous memory–disk storage? (3) Given diverse VSS query tasks (e.g., ANN, range search, and filtered ANN), which tasks does each method support, and how are they implemented in practice? and (4) How are incremental updates and index maintenance handled in disk-resident settings?

Motivated by these questions, this survey presents a systematic and comprehensive review of disk-resident VSS methods, categorized according to a unified taxonomy based on their underlying index structures. The paper is organized in accordance with this taxonomy.

Received 30 January 2026; revised 13 August 2026; accepted 29 August 2026. Date of publication 7 September 2026; date of current version 9 November 2026. This work was supported by the Hong Kong RGC under Grant C2003-23Y, Grant 12202024, and Grant 12201925. Recommended for acceptance by X. Liu. *(Corresponding author: Jianliang Xu.)*

Yitong Song, Huiling Li, Zheng Wu, Lanjing Yi, Bojian Zhu, Xin Huang, and Jianliang Xu are with Hong Kong Baptist University, Hong Kong, SAR, China (e-mail: ytsong@comp.hkbu.edu.hk; cshlli@comp.hkbu.edu.hk; cszhengwu@comp.hkbu.edu.hk; csljyi@comp.hkbu.edu.hk; csbjzhu@comp.hkbu.edu.hk; xinhuang@comp.hkbu.edu.hk; xujl@comp.hkbu.edu.hk).

Xuanhe Zhou is with Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhouxuanhe@sjtu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TKDE.2026.3731393, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3731393

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

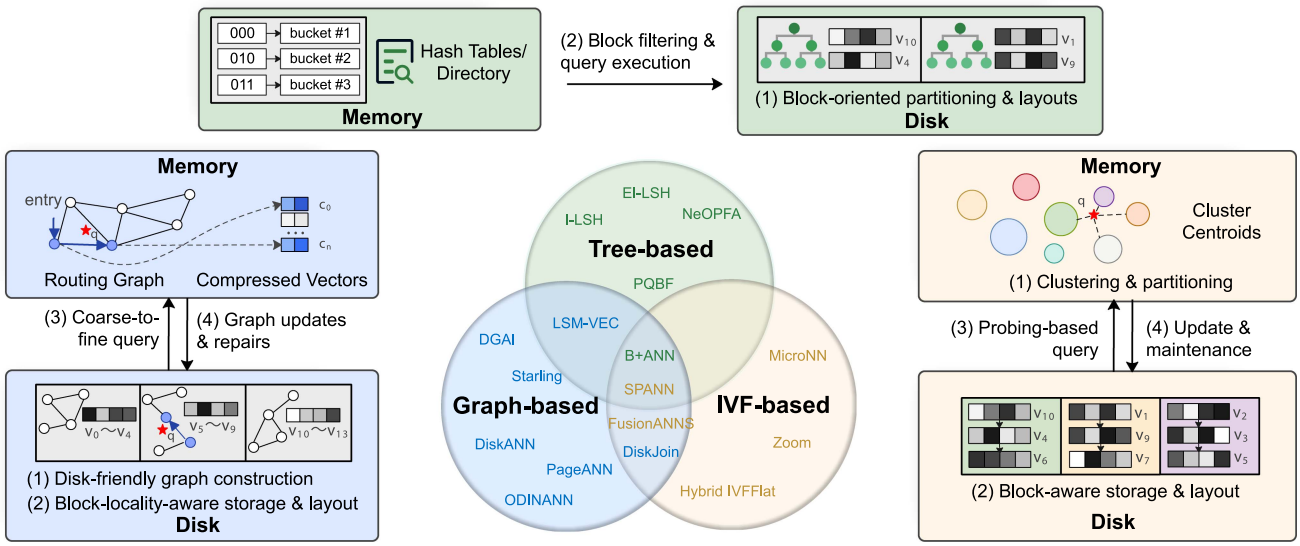



Fig. 1. Illustration of Disk-Resident VSS Methods.

*Taxonomy & Organization:* In contrast to purely memory-resident designs, which often rely on a single index structure, disk-resident VSS methods typically integrate multiple index structures (e.g., graph with quantization [18], [20], or tree with hashing [45], [46]) to support a two-stage filtering pipeline: one for in-memory coarse filtering and another for disk-resident fine-grained filtering. As illustrated in Fig. 1, based on the primary index structures used for filtering, we categorize existing approaches into three groups: *IVF-based methods* (Section III), *graph-based methods* (Section IV), and *tree-based methods* (Section V). Table I summarizes representative methods from each category. Given the distinct design goals and architectural choices across these categories, we further decompose each class into different technical components, as detailed below.

- *IVF-Based Methods:* These approaches partition vectors into clusters and restrict query processing to a small number of relevant clusters [19], [21], [29], [30], [31], [32], [50]. This partition-based design naturally aligns with block-structured storage, enabling efficient block reads and append-friendly updates. However, clustering can incur high memory overhead, while imbalanced clusters complicate block layout and may worsen over time under repeated insertions, degrading query performance. We review IVF-based methods from four aspects: (1) clustering and partitioning (Section III-A), (2) block-aware storage and layout (Section III-B, (3) probing-based query strategies (Section III-C, and (4) update and maintenance mechanisms (Section III-D).
- *Graph-Based Methods:* These approaches organize vectors into proximity graphs and answer queries by traversing these graphs [18], [20], [25], [33], [34], [35], [36], [37], [38], [39], [40], [41], [44]. However, proximity graphs inherently exhibit highly irregular access patterns, and directly placing the entire graph on disk can incur excessive random I/O during query processing. To address this, existing methods redesign graph structures to be more disk-friendly and adopt block-locality-aware layouts to reduce random accesses. They further rely on memory-resident components, such as lightweight routing graphs and compressed vectors, to perform coarse filtering before accessing fine-grained graph neighbors and raw vectors on disk. Moreover, maintaining graph quality under updates is challenging, as deletions often break connectivity and impair query performance. We survey graph-based techniques along four dimensions: (1) disk-friendly graph construction (Section IV-A), (2) block-locality-aware storage and layout (Section IV-B), (3) coarse-to-fine query processing (Section IV-C), and (4) graph updates and repairs (Section IV-D).
- *Tree-Based Methods:* These methods organize vectors into disk blocks using B$^+$-tree structures [48], [49], which are often combined with hash functions [45], [46] or learned models [47] to map high-dimensional vectors into one-dimensional keys. This block-oriented organization enables queries to efficiently locate a small set of candidate blocks via key lookups or range scans, followed by in-block filtering using distances computed from compressed or full-precision vectors. We summarize tree-based techniques along two aspects: (1) block-oriented partitioning and layouts (Section V-A), and (2) block filtering and query execution (Section V-B).

*Comparison & Contributions:* While several excellent surveys on approximate nearest neighbor (ANN) search exist [51], [52], [53], [54], [55], they primarily address memory-resident algorithms, where I/O costs and storage layouts are not central design concerns. Surveys on vector databases [56] provide broader system-level overviews, but devote limited attention to disk-resident indexing techniques. More recently, empirical studies [57], [58] have examined disk-resident graph-based ANN methods through experimental benchmarking and design-space analysis. In contrast, our survey provides broader coverage of disk-resident VSS, spanning ANN, ARS, and FANN queries as well as IVF-based, graph-based, and tree-based methods. We further systematically analyze their key design components and

TABLE I
COMPARISON OF REPRESENTATIVE DISK-RESIDENT VSS METHODS

| Type | Method | Key Techniques | Storage | | Query Type | | | Up.-Opt. | Applications |
|---|---|---|---|---|---|---|---|---|---|
| | | | Memory | Disk | ANN | RS | FANN | | |
| IVF-based | SPANN [19] / SPFresh [21] | Hierarchical balanced clustering / Lightweight rebalancing | IVF centroids + routing graph | IVF lists + vectors | ✓ | × | × | ✓ | Dynamic workloads / frequent updates |
| | MicroNN [29] | On-device lightweight cluster reorganization | IVF centroids | IVF lists + vectors | ✓ | × | ✓ | ✓ | |
| | Zoom [30] | In-memory approximate search & SSD exact reranking | Com. vectors + IVF centroids + IVF lists | Vectors | ✓ | × | × | × | |
| | Hybrid IVFFlat [31] | IVFFlat with advanced filtering | IVF centroids + filter structures | IVF lists + vectors | ✓ | × | ✓ | × | |
| | FusionANNS [32] | CPU–GPU cooperative ANN processing | IVF lists + com. vectors (GPU memory) + routing graph | Vectors | ✓ | × | × | × | |
| Graph-based | DiskANN Series [18, 33, 34, 35] | Vamana graph / Query-sensitive entry / Isomorphic layout / Delta-overlay graph | Com. vectors + entry nodes | Graph + vectors | ✓ | × | ✓ | ✓ | Latency-critical, high-accuracy search |
| | Starling [20] | Segment-based graph partitioning | Com. vectors + routing graph | Vamana + vectors | ✓ | ✓ | × | × | |
| | OdinANN [36] | Direct insert for stable search | Com. vectors + routing graph | Graph + vectors | ✓ | × | × | ✓ | |
| | LSM-VEC [37] | LSM-style multi-level index | Upper layer of HNSW | Bottom layer of HNSW | ✓ | × | × | ✓ | |
| | PageANN [25] | Page-aligned graph | Routing graph | Graph + Com. vectors + vectors | ✓ | × | × | × | |
| | DiskJoin [38] | SSD-oriented vector similarity join processing | Bucket centroids + graph | Vectors | × | ✓ | ✓ | × | |
| | BatANN [39] | Distributed query-state baton passing | Com. vectors + head index | Graph + vectors | ✓ | × | × | × | |
| | PipeANN [40] | I/O-driven I/O-Compute overlapping | Com. vectors + routing graph | Graph + vectors | ✓ | × | × | × | |
| | DGAI [41] | Decoupled graph and vector storage | Com. vectors | Graph + vectors | ✓ | × | × | ✓ | |
| | VeloANN [42] | Affinity-based layout / Coroutine-based search | Hot records (buffer pool) | Graph + Com. vectors + vectors | ✓ | × | × | × | |
| | AlayaLaser [43] | SIMD-friendly layout / Adaptive I/O concurrency | High-degree nodes + entry nodes | Graph + Com. vectors + vectors | ✓ | × | × | × | |
| | FlashANNS [44] | GPU-driven dependency-relaxed asynchronous I/O | Entry nodes | Graph + vectors | ✓ | × | × | ✓ | |
| Tree-based | I-LSH [45] / EI-LSH [46] | Incremental LSH with aggressive early termination | Hash tables | Multi-$B^+$-tree + vectors | ✓ | × | × | × | Low-to-medium dimensions |
| | NeOPFA [47] | Learned sorted-lists with sequential page access | Hash tables | Sorted lists (indexed by $B^+$-tree) + vectors | ✓ | × | × | × | |
| | PQBF [48] | PQ-based filtering with tree pruning | PQ metadata + directory | Com. vectors + PQ$B^+$-tree | ✓ | × | × | × | |
| | B+ANN [49] | Blocked $B^+$-tree with semantic clustering and hybrid traversal | Upper-level $B^+$-tree nodes + block metadata | Blocked $B^+$-tree leaves + graph + vectors | ✓ | × | × | ✓ | |

* RS, FANN, and Up.-Opt. denote range search, filtered ANN search, and whether the method offers optimized update support, respectively.

discuss specialized settings such as distributed and hardware-accelerated search. To the best of our knowledge, no prior work provides such a comprehensive and systematic treatment of disk-resident VSS. Our main contributions are summarized as follows:

- *Dedicated and Systematic Coverage of Disk-Resident VSS:* We present the first survey that focuses exclusively on disk-resident VSS methods, systematically clarifying their query settings, design objectives, and the fundamental I/O challenges that distinguish them from memory-resident approaches.
- *Taxonomy-Driven Analysis of Methods and Design Components:* We classify existing disk-resident VSS methods into IVF-based, graph-based, and tree-based categories, and decompose each category into key technical components, including index construction, block-aware storage layouts, query execution strategies, and update and maintenance mechanisms.
- *Reproducible Benchmarking Resources:* We consolidate commonly used datasets from public sources to enable fair benchmarking of disk-resident VSS methods.
- *Open Challenges and Future Research Directions:* We identify key open challenges and outline several promising research directions for advancing disk-resident VSS.

*Roadmap:* The remainder of this paper is organized as follows. Section II introduces the core concepts of VSS, including common query tasks and widely used underlying techniques. Section III examines IVF-based methods, highlighting how cluster partitioning, block-aligned layouts, probing strategies, and update mechanisms are designed to improve I/O efficiency. Section IV presents graph-based methods, focusing on disk-friendly graph redesigns, locality-aware layouts, coarse-to-fine

searches, and update/repair mechanisms. Section V analyzes tree-based approaches, showing how directory structures and key-based partitioning enable efficient block localization and selective on-disk filtering. Section VI provides an overview of commonly used datasets and their publicly available sources. Section VII discusses open challenges and future research opportunities, and Section VIII concludes the paper.

## II. Preliminaries

This section first outlines the core query problems addressed by VSS methods, and then surveys the key techniques employed in disk-resident VSS, along with their respective advantages and limitations.

### A. Core Query Types

Disk-resident VSS systems mainly address three fundamental query types: *Approximate Nearest Neighbor* (ANN), *Approximate Range Search* (ARS), and *Filtered Approximate Nearest Neighbor* (FANN). The formal definitions of these query tasks are presented below.

*Approximate Nearest Neighbor (ANN):* ANN search is the most commonly used query type in VSS systems. Before defining the ANN problem, we first introduce the definition of $k$ nearest neighbor ($k$NN) problem, which serves as its foundation. Given a finite set of vectors $\mathcal{D}$, a query vector $q$, and an integer $k$, the goal of $k$NN is to retrieve $k$ vectors in $\mathcal{D}$ that are most similar to $q$ under a similarity (or distance) measure $\Gamma$. Formally, $k$NN returns a subset $\mathcal{R}_{knn} \subseteq \mathcal{D}$ such that $|\mathcal{R}_{knn}| = k$ and for all $v \in \mathcal{R}_{knn}$ and $o \in \mathcal{D} \setminus \mathcal{R}_{knn}$, it holds that $\Gamma(v, q) \leq \Gamma(o, q)$. Common choices for $\Gamma$ include Euclidean distance, cosine distance, angular distance, and (negative) inner product, among others.

However, in high-dimensional spaces, exact $k$NN search becomes extremely costly due to the curse of dimensionality [15], [16]. Consequently, recent studies [15], [16], [18], [20], [59] focus on ANN, which relaxes exactness and significantly improves query efficiency. The accuracy of an ANN query is commonly evaluated using the query recall ($Recall@k$), which measures the proportion of true nearest neighbors that are successfully retrieved. Formally,

$$Recall@k = \frac{|\mathcal{R}_{knn} \cap \mathcal{R}_{ann}|}{k},$$

where $\mathcal{R}_{knn}$ and $\mathcal{R}_{ann}$ represent the exact and approximate result sets, respectively.

*Approximate Range Search (ARS):* ARS is another basic query for VSS systems. Unlike $k$NN search, which returns a fixed number of neighbors, range search (RS) focuses on identifying all vectors within a specified similarity radius of the query. Mathematically, RS returns a subset $\mathcal{R}_{rs} \subseteq \mathcal{D}$ such that $\forall v \in \mathcal{R}_{rs}, \Gamma(v, q) \leq radius$. Similarly, exact RS is computationally expensive in high-dimensional spaces [20]. Therefore, ARS is proposed to return an approximate subset of in-range vectors, trading completeness for query efficiency. The accuracy of ARS is typically evaluated by the average precision ($AP@e\%$), which measures the proportion of true in-range vectors retrieved. The precision is defined as

$$AP@e\% = \frac{|\mathcal{R}_{ars}|}{|\mathcal{R}_{rs}|},$$

where $\mathcal{R}_{rs}$ and $\mathcal{R}_{ars}$ represent the exact and approximate result sets, respectively, and $e\%$ indicates the percentage of vectors that fall within the query range.

*Filtered Approximate Nearest Neighbor (FANN):* FANN search extends the ANN search by incorporating attribute-based filtering. Each data object consists of a high-dimensional vector component and an associated scalar attribute (e.g., label, timestamp, and price). The exact FANN query (denoted as FNN) aims to retrieve $k$ objects whose attributes satisfy a given filter condition and whose vectors are most similar to the query vector. Formally, let $\mathcal{D}$ be a dataset of $n$ objects $\{o_1, o_2, \ldots, o_n\}$, where each object $o$ has a scalar attribute $o.a$ and a vector attribute $o.v$. Given a query $Q = (q, \mathcal{A}, k)$ consisting of a query vector $q$, an attribute constraint set $\mathcal{A}$, and an integer $k$, FNN returns a subset $\mathcal{R}_{fnn}$ of $\mathcal{D}$ satisfying: (1) $\mathcal{R}_{fnn} \subset \mathcal{R}_a$, where $\mathcal{R}_a = \{o | o \in \mathcal{D}, o.a \in \mathcal{A}\}$, and (2) $\forall o \in \mathcal{R}_{fnn}, \forall u \in \mathcal{R}_a \setminus \mathcal{R}_{fnn}, \Gamma(o.v, q) < \Gamma(u.v, q)$. Note that If $|\mathcal{R}_a| < k, \mathcal{R}_{fnn}$ contains all data objects in $\mathcal{R}_a$. FANN returns an approximate result set $\mathcal{R}_{fann}$ that trades query accuracy for query efficiency. Its accuracy is commonly measured using the query recall, defined as

$$Recall@k = \frac{|\mathcal{R}_{fnn} \cap \mathcal{R}_{fann}|}{\min(k, |\mathcal{R}_{fnn}|)}.$$

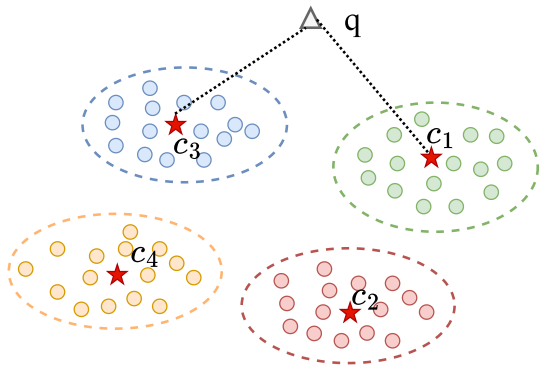



Fig. 2. The IVF Index.

### B. Underlying Techniques

Disk-resident VSS methods accelerate queries through a variety of indexing techniques, most notably *Inverted File* (IVF), *Quantization*, *Proximity Graphs* (PG), and *Locality-Sensitive Hashing* (LSH). In the following, we review these techniques and discuss their respective advantages and limitations.

*Inverted File (IVF):* As illustrated in Fig. 2, the IVF technique partitions the entire vector dataset via clustering, and then maintains an inverted list (also known as posting list) for each cluster to record the vectors assigned to it. Common clustering methods include $k$-means [60] and its variants [61], [62], [63] that address issues such as partition imbalance and stability.

During ANN query processing, the query vector $q$ is first compared to all cluster centroids to identify the $nprobe$ nearest

clusters. Only the vectors within these selected clusters are then scanned to compute their distances to $q$. The $k$ vectors with the smallest distances are returned as the final result. The parameter $nprobe$ controls the search granularity: a smaller value of $nprobe$ reduces the scan size but may impair query accuracy, while increasing $nprobe$ improves accuracy at the cost of query latency.

Moreover, IVF structures are often extended into *hierarchical* or *multi-level* forms, where each coarse cluster is further partitioned into multiple finer-grained sub-clusters [19], [64]. Rather than relying on a single flat clustering layer, these designs introduce a coarse-to-fine clustering hierarchy that progressively narrows the candidate set before accessing disk-resident data blocks.

► *Advantages and Limitations:* IVF provides only coarse-grained pruning, since vectors in the probed clusters must still be scanned, making its query efficiency generally lower than finer-grained approaches such as graph-based methods. Nevertheless, IVF remains widely used in disk-resident VSS because it aligns well with block-based storage. By grouping similar vectors into the same cluster, it improves I/O locality and reduces scattered disk accesses. Its inverted-list structure also naturally supports *dynamic data maintenance:* insertions can append vectors to cluster lists, while deletions require only lightweight list updates. However, repeated updates may gradually cause cluster imbalance and degrade query efficiency, motivating *adaptive partitioning* and *incremental re-clustering* to maintain balanced clusters and stable performance.

*Quantization:* Quantization is a fundamental technique for vector compression in disk-resident VSS methods. It encodes high-dimensional vectors into compact representations that can be stored in memory. Using these compact codes, the system can approximate distances to the query vector without accessing the raw vectors, allowing data vectors that are evidently dissimilar to be filtered out before any disk I/O occurs. This significantly reduces the amount of data that must be read from storage and thus lowers query latency.

Existing quantization approaches can be broadly categorized into vector quantization (VQ) [65], scalar quantization (SQ) [13], [66], [67], and product quantization (PQ) [12], [68]. Among these, PQ has become the dominant choice in disk-resident VSS, as it provides a favorable balance between compression efficiency, query accuracy, and distance computation cost. We therefore describe PQ in detail below.

As shown in Fig. 3, PQ first partitions the original $d$-dimensional space into $m$ subspaces, and learns a codebook by clustering the sub-vectors in each subspace into $C$ groups and storing their centroids. Each vector $x$ is then encoded as an $m$-dimensional code, where the $i$-th entry (an integer from 1 to $C$) indicates which centroid in the $i$-th subspace best represents the corresponding sub-vector.

During query processing, the query vector $q$ is similarly partitioned, and a distance lookup table is constructed by computing the distances between each sub-vector of $q$ and the $C$ centroids in its corresponding subspace. The distance between $q$ and any encoded vector is then estimated by summing the $m$ precomputed lookup values associated with its code. Finally, the

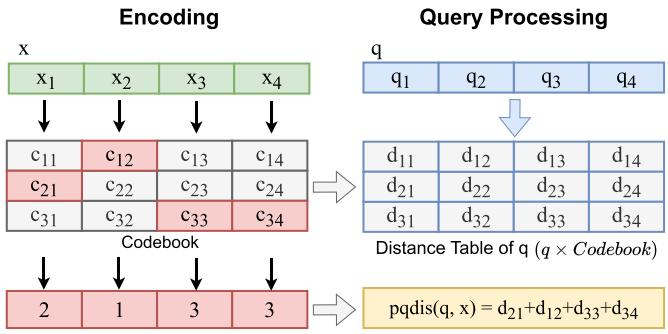



Fig. 3. Product quantization (PQ).

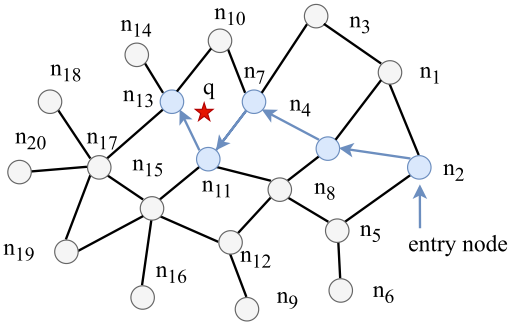



Fig. 4. Proximity Graph (PG).

$k$ vectors with the smallest estimated distances are returned as the query results.

► *Advantages and Limitations:* Compared to directly computing full $d$-dimensional distances, PQ substantially accelerates query processing by reducing costly distance computations to lightweight table lookups. However, quantization inevitably introduces approximation error, which can lead to noticeable recall degradation. In addition, quantization only speeds up distance computation and do not reduce the number of vectors examined during search. Therefore, it is commonly combined with other index structures (e.g., IVF and graph) that provide search-space pruning. Typical hybrid designs include IVFPQ [68], HNSWPQ [27], [69], [70], IVFRaBitQ [13], among others. These methods typically use IVF or graph-based traversal to first narrow the candidate set, and then apply quantization-based distance estimation to efficiently evaluate the remaining vectors.

*Proximity Graph (PG):* As illustrated in Fig. 4, PG organizes vectors as nodes in a graph, where each node is connected to a selected set of nearby vectors. Rather than simply linking each node to its $k$ nearest neighbors, PG selects neighbors from diverse directions within the local neighborhood, which helps avoid overly dense cliques and improves reachability during graph traversal.

To construct a PG, a heuristic two-stage procedure is commonly used. For each node, the algorithm first identifies a larger pool of candidate neighbors (controlled by the parameter $efConstruction$), and then prunes them to retain at most $m$ edges. The pruning strategy prefers neighbors that provide complementary directional coverage, removing those that lie in similar directions to already selected neighbors.

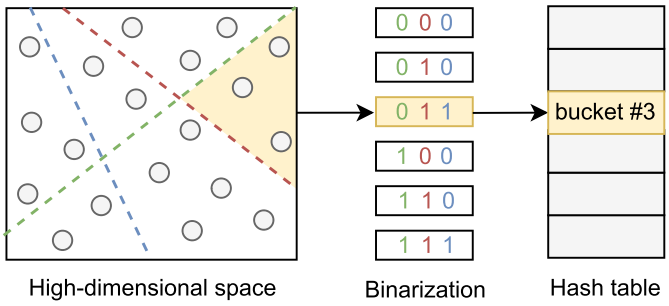



Fig. 5. Locality-Sensitive Hashing (LSH).

During query processing, the search follows a best-first traversal strategy while maintaining a dynamic candidate set $\mathcal{S}$ of size $ef$ ($ef \geq k$). The search is initialized from an entry node and iteratively expands the closest unvisited node in $\mathcal{S}$. Upon expansion, the neighbors of the selected node are evaluated and inserted into $\mathcal{S}$, after which $\mathcal{S}$ is updated to retain only the $ef$ closest candidates to the query $q$. The process continues until no closer candidate can be found. Finally, the top-$k$ nodes in $\mathcal{S}$ are returned as the query result. The parameter $ef$ controls the accuracy-efficiency tradeoff: larger $ef$ generally improves recall but increases search cost.

► *Advantages and Limitations:* Thanks to the elaborated graph structures, PG-based methods generally achieve the best trade-off between query efficiency and accuracy among all types of VSS methods [51], [53]: it can reach high query accuracy with relatively few distance calculations. However, PG structure is memory-intensive, which limits scalability when the dataset grows. When the graph is placed on disk, its inherently random access pattern introduces a large number of random I/O operations, significantly degrading performance. In addition, graph-based indexes are inherently less friendly to dynamic updates. Insertions require costly neighbor re-evaluation, and deletions may degrade graph quality if the graph is not properly repaired, making efficient online maintenance an open challenge.

*Locality-Sensitive Hashing (LSH):* As illustrated in Fig. 5, LSH partitions the high-dimensional vector space into multiple sub-regions using hash functions, where each sub-region corresponds to a hash bucket identified by a hash code. Each vector is mapped to a bucket according to the region in which it falls. These hash functions are designed to preserve locality, such that vectors close in the original metric space are likely to be mapped into the same bucket. Formally, given a metric space with distance function $\Gamma$, a hash function $h$ is said to be $(r_1, r_2, p_1, p_2)$-sensitive if for any two data points $x$ and $y$:

- If $\Gamma(x, y) \leq r_1$, then $\Pr[h(x) = h(y)] \geq p_1$,
- If $\Gamma(x, y) \geq r_2$, then $\Pr[h(x) = h(y)] \leq p_2$.

During query processing, the same hashing procedure is applied to the query vector, and the search is restricted to buckets with hash codes close to that of the query, typically measured by Hamming distance [71].

► *Advantages and Limitations:* LSH-series methods offer strong theoretical guarantees and can retrieve approximate nearest neighbors within a factor of $(1 + \epsilon)$ of the true nearest neighbor distance with high probability. However, achieving high query accuracy typically requires long hash codes, which leads to an exponential increase in the number of buckets. This results in highly sparse bucket occupancy and many empty buckets due to the curse of dimensionality, significantly reducing search efficiency. Consequently, hash-based methods often struggle to maintain both scalability and accuracy in large-scale, high-dimensional search tasks.

## III. IVF-Based Methods

This section introduces IVF-based disk-resident VSS methods, whose query processing workflow is illustrated in Fig. 6. These methods process a VSS query by first identifying a small set of nearest clusters, then retrieving the corresponding vectors from page-aligned secondary storage, and finally computing exact distances to produce the final results. We review existing techniques from four key aspects: (1) *Clustering and Partitioning* (Section III-A); (2) *Block-Aware Storage and Layout* (Section III-B); (3) *Probing-Based Query Strategies* (Section III-C); and (4) *Update and Maintenance Mechanisms* (Section III-D).

### A. Clustering and Partitioning

A standard IVF index trains cluster centroids and assigns each vector to an inverted list. At billion scale, this process faces three main challenges: (1) centroid training and assignment are costly under limited memory; (2) conventional clustering can produce highly imbalanced lists, causing uneven disk I/O and high tail latency; and (3) single-cluster assignment may reduce recall when only a few lists are probed. Accordingly, we review memory-efficient clustering, balanced partitioning, and boundary vector replication techniques to address these challenges.

*Memory-Efficient Clustering:* Standard $k$-means clustering [72] typically requires multiple passes over the dataset and sufficient memory to hold the training data, which becomes a major bottleneck at billion scale. For example, SPFresh [21] reports substantial resource overhead for global IVF rebuilds, highlighting the impracticality of full reconstruction at large scale. To operate under tighter memory budgets, MicroNN [29] and Hybrid IVFFlat [31] adopt *mini-batch k-means* [73], which updates centroids using small batches streamed from disk. This substantially reduces peak memory consumption and enables index construction on modest hardware. However, mini-batch training may slightly degrade clustering quality, which systems typically compensate for through more aggressive probing or additional query-time filtering.

*Balanced Partitioning:* Classical $k$-means minimizes quantization error but does not control cluster sizes, which can create a few oversized inverted lists that dominate disk query latency. A common solution is to introduce balance constraints or size penalties during assignment, e.g.,

$$\min_{C,P} \sum_{x \in X} \|x - c_{P(x)}\|^2 \; + \; \lambda \sum_{i=1}^{N_{\text{list}}} \left(|X_i| - |X|/N_{\text{list}}\right)^2,$$

where $|X_i|$ denotes the inverted-list length, and the second term penalizes oversized lists. SPANN [19] explicitly addresses disk-side cluster imbalance by producing many small inverted lists

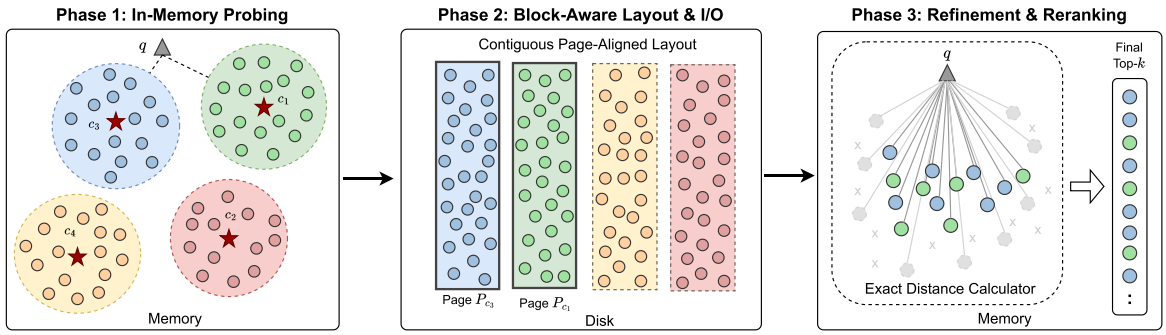



Fig. 6. Query workflow of IVF-based methods.

with tightly controlled sizes through hierarchical balanced clustering. MicroNN [29] pursues the same goal with a lightweight, balance-aware clustering pipeline compatible with mini-batch training. For dynamic workloads, SPFresh [21] maintains balanced SPANN-style partitions under inserts/deletes by splitting oversized lists, merging underfull ones, and locally reassigning nearby vectors to restore nearest-partition assignment. In contrast, Zoom [30] and Hybrid IVFFlat [31] use standard $k$-means without explicit balance constraints, instead tuning the number of clusters ($N_{\text{list}}$) and probing budgets to balance scan cost and accuracy.

*Boundary Vector Replication:* When only a small number of clusters are probed, true nearest neighbors may fall just across cluster boundaries and be excluded from the search, leading to recall loss. Boundary vector replication offers an effective mechanism to mitigate this issue. SPANN [19] implements this idea via *closure assignment*, where a vector may be stored in multiple nearby inverted lists if it is nearly equidistant to several centroids. A typical replication rule is:

$$d(x, c_{i_j}) \leq (1+\varepsilon)\, d(x, c_{i_1}), \quad j = 1, \ldots, R_{\max},$$

which ensures that only boundary vectors are duplicated, subject to a small replication cap. While this strategy increases index size, it significantly improves query recall under a fixed probing budget. FusionANNS [32] adopts a similar boundary replication criterion during clustering, but subsequently modifies which data are actually placed on disk. In contrast, MicroNN [29], Zoom [30], and the Hybrid IVFFlat [31] typically rely on probing more centroids (i.e., increasing $n_{\text{probe}}$) to recover recall, which may increase scan cost.

### B. Block-Aware Storage and Layout

An IVF-based disk-resident VSS method typically maintains three core components: cluster centroids, inverted lists, and raw vectors. As summarized in Table II, existing methods mainly adopt two memory–disk organizations. The first keeps only centroids and lightweight metadata in memory while storing inverted lists and raw vectors on SSD, reducing memory usage but limiting pre-I/O pruning. The second additionally keeps inverted lists and compact vector representations (e.g., PQ codes) in memory, enabling stronger pruning at higher memory cost. FANN-oriented methods may further co-locate filtering attributes with vectors to improve access locality.

TABLE II
TYPICAL MEMORY–DISK ORGANIZATIONS IN IVF-BASED METHODS

| | IVF Lists on Disk | IVF Lists in Memory |
|---|---|---|
| **Memory** | Centroids, Metadata | Centroids, IVF lists, Com. codes |
| **Disk** | IVF lists, raw vectors | Raw vectors |
| **Tradeoff** | Lower memory cost<br>Limited pre-I/O pruning | Higher memory cost<br>Stronger pre-I/O pruning |
| **Methods** | SPANN [19], SPFresh [21], MicroNN [29] | Zoom [30], FusionANNS [32] |

*IVF Lists on Disk:* SPANN [19] keeps cluster centroids and a lightweight routing structure (i.e., SPTAG) in memory, while storing inverted lists and raw vectors on SSD as contiguous arrays. SPFresh [21] preserves this "centroids-in-memory / lists-on-SSD" design and augments it with lightweight metadata for efficient in-place updates. MicroNN [29] similarly retains only IVF centroids in memory, but stores vectors in a relational store (SQLite) using a clustered key (e.g., partition ID), such that scanning an list becomes a near-sequential range scan.

*IVF Lists in Memory:* Zoom [30] keeps centroids, inverted lists, and PQ codes in memory while storing raw vectors on disk. Its in-memory preview stage filters candidates using PQ codes, after which only a small number of raw vectors are fetched for exact reranking. FusionANNS [32] extends this design with a *CPU–GPU–SSD* split: the centroid routing graph and vector IDs reside in CPU memory, PQ-compressed vectors in GPU HBM, and raw vectors on SSD. Only surviving candidates are fetched from SSD for exact distance computation.

*Hybrid IVF Lists with Filtering Attributes:* Hybrid IVFFlat [31] stores raw vectors and filtering attributes as unified "hybrid vectors" within each inverted list on SSD. Candidate lists are first selected via centroid-based pruning, after which attribute predicates are applied during list scanning. This co-location improves access locality for FANN queries.

### C. Probing-Based Query Strategies

IVF-based disk-resident VSS methods process queries by probing only a small subset of clusters on disk. Efficiently routing queries to relevant clusters, reducing unnecessary data accesses, and minimizing repeated I/O across queries are critical

to search performance. In the following, we review techniques for cluster probing and dynamic pruning, as well as caching and batching strategies, respectively.

*Cluster Probing and Dynamic Pruning:* A basic IVF implementation compares the query against all cluster centroids to identify the $n_{\text{probe}}$ nearest clusters for probing. To reduce this routing overhead, SPANN [19] builds a lightweight in-memory graph over centroids to accelerate nearest-centroid search. Zoom [30] similarly replaces exhaustive centroid scans with an in-memory HNSW-based routing layer. FusionANNS [32] also employs a centroid graph for fast list selection, but further relies on GPU-side PQ filtering to keep the subsequent SSD access lightweight.

Beyond routing acceleration, SPANN [19] introduces a *query-aware dynamic pruning* rule to reduce the amount of data read from SSD during probing. After identifying the closest $K$ centroids, an inverted list is scanned only if its centroid is sufficiently close to the nearest one:

$$q \xrightarrow{\text{scan}} X_{ij} \iff \text{Dist}(q, c_{ij}) \le (1+\varepsilon)\,\text{Dist}(q, c_{i1}),$$
$$\text{Dist}(q, c_{i1}) \le \cdots \le \text{Dist}(q, c_{iK}).$$

This pruning criterion avoids unnecessary inverted-list reads for "easy" queries while preserving high recall. In contrast, OrchANN [74] applies traditional triangle-inequality–based pruning at the vector level to reduce the number of raw vectors read from disk.

*Caching and Batching:* To reduce repeated and unnecessary I/O, disk-resident IVF methods commonly use caching and batching optimizations. MicroNN [29] largely relies on the database buffer cache, making performance sensitive to cache warmness; it also supports batch execution that groups queries by partition and scans each partition once, amortizing both I/O and distance computation. Hybrid IVFFlat [31] uses frequently accessed lists as an on-demand cache to avoid repeated disk accesses to hot inverted lists. Zoom [30] reduces SSD reads by moving most candidate generation to an in-memory preview stage, where PQ codes are scanned to produce a small candidate set and only these candidates are fetched from SSD for exact reranking. FusionANNS [32] adopts a similar design with GPU-side PQ filtering followed by lightweight SSD reranking. It further reduces reranking cost by (1) processing candidates in mini-batches with early termination once the top-$k$ results stabilize, and (2) reducing read amplification by deduplicating and merging I/Os to the same SSD pages while reusing the memory buffer across mini-batches.

### D. Update and Maintenance Mechanisms

In real-world applications, vector datasets are highly dynamic, with frequent insertions and deletions. For IVF indexes, updates are naturally supported by simply appending or removing vectors from the cluster lists. However, as updates accumulate, inverted lists may become imbalanced and cluster centroids may drift, which can degrade both routing accuracy and I/O efficiency. Accordingly, existing methods address this challenge from two complementary aspects: efficient update support and incremental rebalancing mechanisms.

*Update Support Strategies:* Several disk-resident IVF methods explicitly address update efficiency at the storage layer. MicroNN [29] leverages the transactional capabilities of its underlying relational database to support atomic inserts and deletes, delegating concurrency control and crash recovery to the storage engine. SPFresh [21] enables in-place updates on SSD by maintaining versioned metadata for inverted lists, allowing list contents to be modified without rewriting the entire index.

*Incremental Rebalancing:* Beyond storage-level update support, SPFresh [21] further addresses structural degradation caused by skewed updates through the *Lightweight Incremental RE-balancing (LIRE)* protocol. LIRE monitors inverted list sizes and performs local reorganization when imbalance occurs: oversized lists are split, undersized lists are merged, and affected vectors are locally reassigned to restore nearest-partition consistency.

## IV. Graph-Based Methods

In contrast to IVF-based approaches, whose block-oriented access patterns and lightweight, update-friendly indexes (excluding raw vectors) naturally suit disk storage, graph-based methods have three key limitations: (1) graph traversal causes irregular random accesses to individual vectors, leading to poor I/O locality; (2) complex graph structures incur high memory overhead, making full in-memory residency difficult; and (3) dynamic updates are challenging because deletions may break graph connectivity and degrade query performance.

To address these issues, disk-resident graph-based VSS methods commonly combine graph search with quantization (e.g., PQ). As shown in Fig. 7, a lightweight routing graph and compressed vectors (e.g., PQ codes) are kept in memory, while raw vectors and fine-grained graph structures reside on disk. Existing methods either redesign graphs for disk access or optimize the physical layout of graph data and vectors to improve locality. Query processing typically follows a coarse-to-fine strategy: in-memory routing and PQ codes guide the search, and only a small number of relevant disk blocks are fetched for refinement. To support dynamic updates, graph repair and maintenance mechanisms are further introduced.

In this section, we review graph-based disk-resident VSS methods along four dimensions: (1) *disk-friendly graph construction* (Section IV-A), (2) *block-locality-aware storage and layout* (Section IV-B), (3) *coarse-to-fine query processing* (Section IV-C), and (4) *graph updates and repair mechanisms* (Section IV-D).

### A. Disk-Friendly Graph Construction

This section introduces graph structures specifically designed for disk-resident settings and discusses construction under limited memory.

*Disk-Based Graph Structures:* As summarized in Table III, existing graph structures can be broadly divided into *neighbor-oriented* and *page-oriented* designs. The former improves graph navigability through neighbor pruning or diversification, while the latter explicitly incorporates disk-page locality into graph

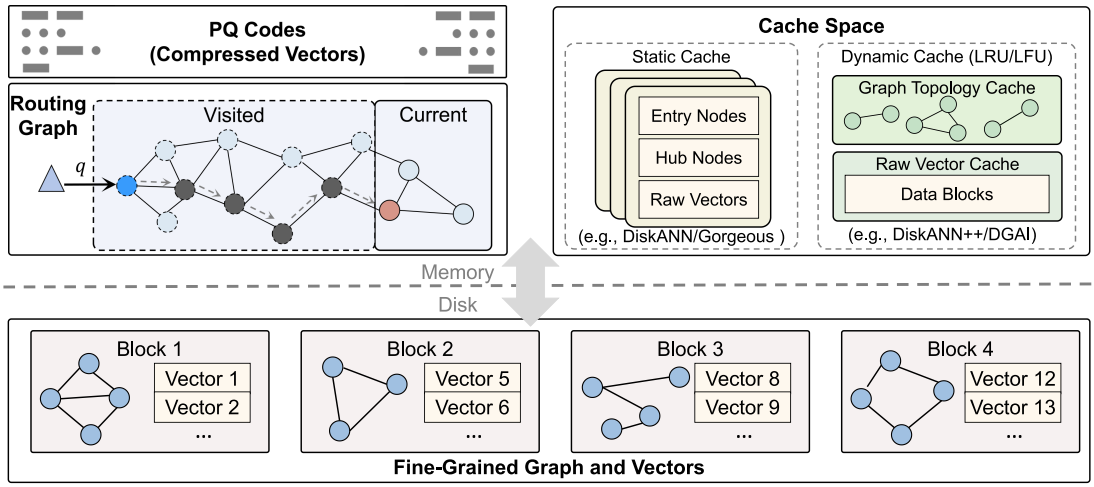



Fig. 7. Illustration of graph-based methods.

TABLE III
SUMMARY OF DISK-ORIENTED GRAPH STRUCTURES

| Type | Method | Construction Focus |
|---|---|---|
| Neighbor-oriented | Vamana [18] | Neighbor pruning |
| | XN-Graph [24] | Neighbor diversification |
| Page-oriented | BAMG [22] | Block locality |
| | PageANN [25] | Node organization |

construction. Representative methods include Vamana [18], XN-Graph [24], BAMG [22], and PageANN [25].

- *Vamana* [18] constructs a sparse proximity graph using a pruning rule with a tunable parameter $\alpha > 1$ that controls the pruning process by retaining only those neighbors whose inclusion results in a sufficiently large improvement in distance to the query. It also enforces a fixed out-degree to facilitate block-aligned storage. Compared with in-memory graphs like NSG [16] and HNSW [15], these design choices often lead to fewer traversal hops and disk round trips, making Vamana well suited for disk-based search.
- *XN-Graph* [24] is a disk-oriented graph index inspired by NN-Descent [75]. It iteratively collects diversified intermediate neighbors from a wider coverage of the data space, rather than relying on neighbors encountered along traverse paths in prior methods [16], [18]. This design helps the graph provide multiple alternative paths during search.
- *BAMG* [22] proposes a block-aware edge selection strategy that differentiates between intra-block and inter-block connections. Unlike Vamana, BAMG focuses on optimizing the per disk access progress, preferentially retaining edges that improve local navigability without incurring additional I/O operations.
- *PageANN* [25] groups multiple similar vectors into a single unit that fits within one disk page, referred to as a *page node*. Search is then performed over these page nodes, so that each traversal step corresponds to reading a disk page rather than accessing individual vectors. For graph construction, PageANN derives links among page nodes from a high-quality graph index (e.g., Vamana), while eliminating redundant edges and removing links among nodes within the same node.

*Partition-Then-Merge Construction:* When the entire dataset cannot fit in memory, constructing a graph index becomes challenging, as graph construction typically requires repeated neighbor comparisons and sufficient in-memory context. A widely adopted solution is the *partition-then-merge* paradigm, which avoids holding the full dataset in memory by dividing it into smaller partitions and constructing the graph incrementally. Under this paradigm, the dataset is first partitioned into multiple subsets that can fit into memory. A graph is then constructed independently within each partition, and the resulting subgraphs are finally merged into a single global graph. DiskANN [18] is a representative example of this approach. It partitions the dataset using k-means [72] clustering and assigns each vector to multiple nearby clusters (e.g., two) to preserve cross-partition connectivity. Within each partition, a Vamana graph is constructed entirely in memory. Afterward, the subgraphs are merged by taking the union of their edges to form the final index.

The partition-then-merge paradigm enables graph construction at billion scale under limited memory budgets. However, the quality of the final graph depends on the effectiveness of both partitioning and merging, and suboptimal connections may arise near partition boundaries.

### B. Block-Locality-Aware Storage and Layout

Existing methods differ significantly in their storage modalities and physical layout strategies. How graph structures and raw vectors are placed and organized directly affects access patterns, cache utilization, and I/O behavior, and ultimately determines query efficiency. In this section, we review memory–disk hybrid storage strategies and block-locality-aware layout designs adopted by existing methods.

*Memory-Disk Hybrid Storage Strategies:* Graph-based disk-resident VSS methods typically involve three types of data to be stored: compressed vectors (e.g., PQ codes), full graph

structures, and raw vectors, while the routing graph is maintained selectively. Depending on whether available memory is sufficient to hold PQ codes for all vectors, existing methods can be broadly categorized into two storage regimes:

- *All-in-Disk*, where PQ codes, raw vectors, and full graph structures are all stored on disk [43], [76], [77];
- *Major-in-Disk*, where raw vectors and graph structures reside on disk with PQ codes in memory. This is the predominant design adopted by most existing methods [18], [20], [24], [33], [34], [35], [78].

The former minimizes memory consumption at the cost of increased disk I/O, whereas the latter reduces I/O overhead by trading for a larger memory footprint. In the following, we review these two types of approaches in turn.

1) *All-in-Disk Methods:* All-in-Disk approaches reduce memory requirements by persisting all major data structures on disk, retaining in memory only minimal state for routing (e.g., lightweight routing structures) and I/O coordination (e.g., buffering). Representative methods include LM-DiskANN [77], which maintains only small routing structures and metadata in memory. Similarly, AiSAQ [76] stores only a tiny amount of fixed metadata, PQ codebooks, and optionally the PQ codes of a few entry points in memory. During search, each hop reads an SSD page containing neighbor identifiers and their PQ codes. The traversal decisions are guided by PQ-based distance estimates computed from these in-chunk codes, while exact distance evaluation is deferred until full vectors are fetched from the same chunk. AlayaLaser [43] similarly follows an all-in-disk design, keeping only lightweight routing information, including cached high-degree nodes and cluster-based entry points, in memory.

2) *Major-in-Disk Methods:* These methods are exemplified by DiskANN [18], which balances memory usage and query performance by keeping PQ codes in memory for fast approximate distance estimation, while storing the full graph and raw vectors on disk for fine-grained traversal and final refinement. The PQ codes estimate distances to neighboring nodes and guide traversal toward the most promising one. Its disk block is then fetched to obtain adjacency information for further traversal and raw vectors for exact distance evaluation. This design typically requires only one block read per hop and has become the mainstream approach in graph-based disk-resident VSS [20], [24], [33], [34], [35], [78]. However, its memory footprint grows linearly with dataset size, making it less suitable under extremely tight memory budgets.

While most systems rely on in-memory PQ codes, recent works explore coarser or more compact metadata, such as shard centroids, bucket summaries, or page-aligned routing structures, to further reduce memory usage while preserving effective pruning and traversal. SmartANNS [79], for example, keeps shard centroids, a lightweight GBDT-based pruning model, and minimal scheduling metadata in host DRAM. This information selects likely shards and balances load across SmartSSDs, while fine-grained HNSW traversal and distance computation are offloaded to SSD-side FPGAs. DiskJoin [38], designed for vector joins, stores bucket-level metadata such as centers, radii, and ordering state in memory to determine which bucket pairs

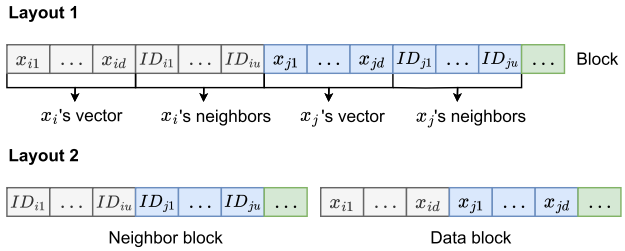



Fig. 8. Two typical data layouts [27].

to verify and in what order, enabling batched, cache-aware disk reads and avoiding unnecessary I/O. Full-precision verification is performed only when needed.

Finally, some systems extend these designs to heterogeneous memory. HM-ANN [80], for instance, places hot data, including frequently accessed nodes and their neighbors, in faster tiers such as DRAM or NVM, while keeping cold data, including raw vectors, in slower tiers.

*Bock-Locality-Aware Layouts:* In graph-based disk-resident VSS, a key challenge is organizing graph indexes, raw vectors, and metadata across disk blocks for efficient I/O. As shown in Fig. 8, existing strategies can be broadly divided into *coupled* layouts [18], [20], [23], [25], [26], which co-locate raw vectors with their graph nodes, and *decoupled* layouts [22], [27], [41], [81], which store adjacency lists and raw vectors separately. Coupled layouts reduce per-hop I/O by fetching adjacency information and raw vectors together, while decoupled layouts allow graph traversal and data access to be optimized independently. We next detail these two strategies.

1) *Coupled Layouts:* Coupled layouts co-locate graph adjacency lists with their corresponding raw vectors within the same disk blocks or pages. This design philosophy is based on the observation that during graph traversal, when a node is accessed, both its vector representation and its adjacency list are typically needed. By storing them together, a single disk I/O operation can retrieve both pieces of information, reducing the total number of I/O operations required per query. However, when adopting coupled layouts that co-locate vectors with their adjacency lists, a key challenge arises: determining which raw vectors and their adjacency lists should share a disk block so as to minimize random I/O during graph traversal. This problem has been proven to be NP-hard [20].

We next review representative approaches to block-aware layout optimization, as summarized in Table IV. DiskANN++ [33] uses isomorphic mapping to pack graph-close nodes into the same SSD page, while Starling [20] directly maximizes neighbor overlap within each block through block shuffling. Li et al. [58] systematically evaluate this strategy and show that block shuffling is more effective when combined with block-aware search. MARGO [23] adopts a monotonic path-aware layout that prioritizes co-locating nodes along important monotonic search paths. PageANN [25] abstracts similar vectors into page nodes to align graph traversal with SSD page accesses. VeloANN [42] combines hierarchical compression with affinity-based data placement to co-locate frequently related vectors within the same

TABLE IV
SUMMARY OF LAYOUT OPTIMIZATION METHODS

| Type | Method | Layout Optimization | |
|---|---|---|---|
| Coupled | DiskANN [18] | Round robin assignment | |
| | DiskANN++ [33] | Isomorphic mapping | |
| | Starling [20] | Block shuffling | |
| | MARGO [23] | Monotonic path-aware packing | |
| | PageANN [25] | Clustering-based | |
| | AlayaLaser [43] | SIMD-friendly layout | |
| **Type** | **Method** | **Neighbor** | **Vector** |
| Decoupled | GaussDB-Vector [81] | Heuristic | Sorted contiguous |
| | DGAI [41] | Similarity-aware | Similarity-aware |
| | TRIM [27] / BAMG [22] | Block shuffling | Block-aligned |

disk page, reducing data fragmentation and read amplification during graph traversal. While these methods primarily optimize I/O locality, AlayaLaser [43] targets computation efficiency in high-dimensional settings by adopting a SIMD-friendly on-disk layout for faster distance computation.

2) *Decoupled Layouts:* Decoupled layouts store graph adjacency lists and raw vectors separately, allowing each component to be organized according to its own access pattern. GaussDB-Vector [81] adopts this design at the database-system level: indexes (IVF or graph-based) reside in dedicated files or pages, while raw vectors are sorted and stored contiguously for sequential disk access. During search, traversal proceeds on the lightweight index, and raw vectors are fetched only when needed. TRIM [27] and BAMG [22] similarly exploit decoupled layouts. BAMG stores the graph index and block metadata in compact structures to reduce I/O, while keeping raw vectors in separate contiguous regions that are accessed in batches during reranking. TRIM separates PQ codes and metadata from full-dimensional vectors, so raw vectors are fetched only for a small set of candidates after pruning. This avoids unnecessary data loading and distance computation while enabling independent, hardware-adaptive optimization of index and data layouts. DGAI [41] further decouples graph topology from vectors to accelerate updates by eliminating redundant I/O. To offset the resulting query overhead, it uses similarity-aware topology reordering to colocate topologically close nodes on the same pages, improving locality and reducing traversal I/O.

### C. Coarse-to-Fine Query Processing

Disk-resident graph search must jointly minimize (1) how far the search traverses (i.e., the number of hops) and (2) how much data it reads per hop (i.e., the number of I/Os per read). A common baseline maintains a small in-memory candidate set, iteratively expands the nearest unvisited node, estimates cheap approximate distances (e.g., via PQ methods) for pruning, fetches on-disk blocks only when necessary, and finally re-ranks a small candidate set using full-precision vectors. In this section, we review the search patterns and execution strategies adopted by existing methods, including in-memory coarse search, disk-based fine search with asynchronous execution, cache management strategies, as well as query optimizations for special scenarios such as distributed search and hardware (e.g., GPUs) acceleration.

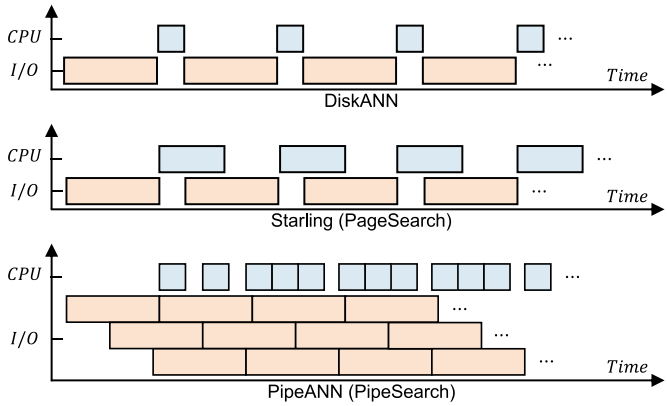



Fig. 9. Execution timeline for DiskANN, starling, and PipeANN.

*In-Memory Coarse Search:* To reduce disk I/Os, existing methods maintain lightweight information in memory for routing, approximate evaluation, or candidate pruning [18], [20], [23], [27], [40], [43], [77], [80], [82]. DiskANN [18] keeps entry points and PQ codes in memory, using approximate distances to guide traversal before fetching full-precision vectors from disk. HM-ANN [80] follows a similar principle in heterogeneous memory architectures, dynamically placing frequently accessed navigation data in faster memory tiers. Starling [20] instead constructs a lightweight in-memory navigation graph to identify better starting points for disk-resident traversal. Li et al. [58] further study this design as *MemGraph*, showing that a lightweight in-memory navigation graph can provide higher-quality entry points and shorten subsequent disk-resident traversal paths. AlayaLaser [43] selects query-specific entry points through in-memory clustering to shorten traversal paths. Beyond routing, TRIM [27] uses lightweight in-memory metadata to derive distance lower bounds and prune unpromising candidates before accessing their full-precision vectors. SkipDisk [82] similarly avoids unnecessary disk accesses by deriving triangle-inequality bounds from dedicated pivots, together with multi-level pruning to control memory overhead.

*Disk-Based Fine Search with Asynchronous Execution:* Disk-resident VSS methods often exploit asynchronous I/O and disk-aware layouts to optimize I/O scheduling during query processing and reduce data access costs. As shown in Fig. 9, DiskANN [18] serves as a baseline method that adopts a synchronous execution pattern, whereas Starling [20] and PipeANN [40] are two representative asynchronous search systems based on PageSearch and PipeSearch, respectively. We then review PageSearch and PipeSearch to illustrate their distinct approaches to execution optimization.

(1) *PageSearch:* PageSearch [20] follows a page-oriented search process in which each fetched page is first processed to update candidate distances and graph expansions, and the resulting candidate set then determines the subsequent pages to access. Since each I/O retrieves an entire page, PageSearch further exploits page locality by processing other promising nodes co-located with the target node, allowing multiple graph expansions to share a single I/O. Starling [20] also asynchronously prefetches promising pages to overlap I/O with computation.

VeloANN [42] follows a similar execution pattern, but uses lightweight coroutines to hide I/O stalls: a search coroutine yields the CPU while waiting for its requested page and resumes upon completion to process the fetched vectors and determine subsequent page accesses. SkipDisk [82] combines in-memory candidate pruning with asynchronous I/O by retaining graph neighbors in memory, allowing candidate generation and filtering to proceed while disk reads are in flight.

(2) *PipeSearch:* PipeSearch [40], in contrast, relaxes the dependency between candidate processing and subsequent I/O. Its key observation is that the current candidate pool already provides useful hints about which records are likely to be accessed next. Rather than waiting for previously issued I/Os and neighbor expansions to complete, PipeSearch proactively issues asynchronous reads for promising unread candidates whenever I/O capacity is available. Meanwhile, previously fetched vectors are processed in a best-effort manner, allowing computation and I/O to proceed concurrently. This pipelined execution keeps the SSD I/O pipeline busy and effectively hides I/O latency.

While PageSearch and PipeSearch improve efficiency by overlapping I/O with computation, existing methods [40], [43], [58] further improve SSD utilization by dynamically adjusting the beam width to control I/O concurrency. PipeANN [40] adjusts the beam width based on the candidate pool state, while AlayaLaser [43] progressively increases it with the search hop count. Li et al. [58] instead adapt the beam width according to the available I/O parallelism. These strategies improve SSD utilization while avoiding unnecessary disk accesses.

*Cache Management Strategies:* Caching is a key component for reducing disk I/O in disk-resident VSS. By keeping frequently accessed data in memory, systems can significantly reduce disk reads and improve query latency. Existing cache management strategies can be broadly categorized into three classes: static, dynamic, and hybrid static–dynamic strategies.

(1) *Static Caching Strategy:* Static caching determines cache contents before query execution, typically during index construction or offline preprocessing. It is effective when access patterns are predictable or when certain objects, such as entry points or hub nodes, are frequently accessed regardless of the query. DiskANN [18], for example, caches frequently visited nodes within a few hops of the graph entry point. LEANN [78] similarly identifies frequently accessed hub nodes and stores them in a dedicated cache region. These strategies mainly rely on historical access patterns rather than future query behavior. In contrast, DiskJoin [38] constructs a predetermined read schedule over bucket pairs to maximize locality and combines it with a Belady-style [83] eviction policy that exploits future access information to minimize cache misses. The cache thus retains only the buckets needed at each step. Recent works [26], [33], [41] further observe that adjacency lists are accessed much more frequently than raw vectors during search. Based on this, Gorgeous [26] prioritizes caching compact adjacency lists, allowing a larger fraction of the graph structure to remain in memory.

(2) *Dynamic Caching Strategies:* Dynamic caching strategies make cache decisions online based on query characteristics, access patterns, and system state, and are particularly effective for query-dependent workloads where optimal cache contents cannot be determined a priori. For example, on heterogeneous memory platforms, HM-ANN [80] promotes frequently accessed high-level adjacency lists to fast DRAM, while placing less frequently used graph regions in slower but higher-capacity NVRAM. As workloads evolve, HM-ANN automatically migrates data across memory tiers to adapt to changing access patterns and improve cache efficiency. DiskANN++ [33] further exploits dynamic caching by maintaining a page heap that enables asynchronous expansion of the search frontier on cached SSD pages while future I/O requests are still in flight. This design both increases cache hit rates for topology pages and hides SSD access latency through overlapping computation and prefetching. Similarly, XN-Graph [24] prioritizes expanding nodes whose neighborhoods are already resident in the cache, rather than waiting for pending SSD reads, and adopts a simple FIFO policy for cache management.

3) *Hybrid Static-Dynamic Caching:* Hybrid caching combines static caching of globally important nodes with dynamic caching of query-dependent accesses. DGAI [41] statically pins entry nodes and their multi-hop neighbors, while dynamically caching recently accessed topological pages. GoVector [28] further exploits vector similarity to proactively load nearby nodes along search paths and uses an LFU replacement policy to retain frequently accessed data. VeloANN [42] uses a finer-grained record-level buffer pool that groups each vector's neighbors into a record, reducing page swapping and improving cache utilization under constrained memory budgets.

*Optimizations for Special Scenarios:* Beyond core query designs, practical graph-based vector search systems must accommodate diverse deployment environments and workload characteristics. In large-scale and performance-critical settings, this often necessitates distributed deployments and hardware acceleration. Accordingly, this section reviews representative optimizations tailored to distributed execution and hardware-accelerated query processing.

1) *Distributed Search:* As vector datasets outgrow a single server, distributed systems [1], [84], [85] are needed for scalable capacity and throughput. Graph-based systems mainly follow two paradigms: (i) partition the dataset and build an independent graph per partition, or (ii) build one global graph and distribute its nodes across servers. In the first, queries follow a scatter–gather workflow: relevant partitions perform local graph search, and a coordinator merges their results. This design is simple and parallelizable, but may lose global graph connectivity and reduce search efficiency. The second preserves a unified graph structure and its search efficiency, but requires efficient traversal across server boundaries. BatANN [39] addresses this with asynchronous state passing, forwarding the query state to the server owning the next graph region so traversal can continue locally with lower cross-server communication overhead.

2) *Hardware Acceleration:* Graph-based VSS is often accelerated using hardware technologies, such as GPUs [44], [86], [87], [88] and SIMD [89], [90]. In disk-resident settings, hardware acceleration is commonly used, particularly for distance computations. FlashANNS [44] introduces a GPU-driven asynchronous I/O framework to address the storage-compute bottleneck. It decouples I/O and distance computation while

TABLE V
SUMMARY OF GRAPH UPDATE AND REPAIR STRATEGIES

| Type | Method | Update Mechanism | Repair Strategy |
|---|---|---|---|
| In-place | OdinANN [36] | Direct on-disk insertion | Local repair |
| | DGAI [41] | Decoupled in-place update | Locality-aware repair |
| Out-of-place | Fresh-DiskANN [34] | Short-term index + StreamingMerge | $\alpha$-RNG local repair |
| | LSM-VEC [37] | Multi-level LSM merging | Local reconstruction |

waiting for I/O responses. The difference is that it leverages GPUs to optimize the distance computation. GustANN [86] focuses on GPU-accelerated batch processing, targeting scenarios where multiple queries need to be processed concurrently. It leverages GPU's parallel processing capabilities to handle distance computations for all queries in the batch simultaneously. This batched approach amortizes the overhead of GPU memory transfers and kernel launches across multiple queries, significantly improving overall system throughput.

### D. Graph Updates and Repairs

Dynamic workloads require continuous updates to both vector data and graph structures, making efficient updates and graph-quality preservation critical for graph-based disk-resident VSS. In this subsection, we review two major update strategies, *in-place* and *out-of-place* updates, together with their graph repair mechanisms. Table V summarizes representative methods by update type, mechanism, and repair strategy.

*Update Strategies:* A primary challenge in dynamic graph indexes is the high I/O cost of index maintenance. In typical coupled layouts, vectors and graph structures share disk pages, so insertions or deletions may require rewriting pages of multiple affected neighbors, causing substantial I/O amplification. Existing methods therefore mainly adopt in-place or out-of-place updates to reduce update overhead.

(1) *In-Place Updates:* In-place update strategies directly modify index structures at their existing disk locations, avoiding deferred writes and bulk reorganization. By eliminating buffering and merge-induced write amplification, they reduce I/O bursts and help stabilize query performance under dynamic workloads. OdinANN [36] adopts a direct-insert policy, immediately inserting each new vector into the on-disk graph without intermediate buffering. This avoids costly merges and reduces interference between update-intensive phases and foreground queries, leading to smoother latency. DGAI [41] further improves update efficiency with a decoupled layout that separates graph structure from vector data, so inserting or adjusting graph connections typically modifies only compact graph pages. It also performs locality-aware in-place updates by placing new nodes near similar neighbors and limiting page splits to local reorganization. Together, these designs reduce page modifications and mitigate update-induced I/O amplification.

(2) *Out-of-Place Updates:* Out-of-place updates write new or modified vectors to a new location (e.g., a short-term layer) rather than overwriting the originals, then asynchronously merge and clean up old versions to keep sequential writes and high throughput. Fresh-DiskANN [34] employs a background *StreamingMerge* process that batches incremental updates from an in-memory short-term index into a long-term SSD-resident index, amortizing costly writes and preserving system responsiveness. LSM-VEC [37] adopts a hierarchical architecture combining in-memory upper layers with a disk-resident bottom layer managed via a log-structured merge (LSM) tree, optimizing write amplification and enabling scalable ingestion of dynamic data. In addition, it stores vectors and graph structures separately, allowing LSM-VEC to update the graph without touching vectors.

*Graph Repair:* Maintaining high query recall under continuous updates is challenging, as naive insertions and deletions can gradually degrade graph connectivity, reduce navigability, and ultimately harm retrieval quality. As a result, effective graph repair mechanisms are essential to restore high-quality neighbor links and preserve query recall over time. Fresh-DiskANN [34] addresses this challenge by introducing *FreshVamana*, a graph index that proactively maintains graph quality during updates, avoiding expensive periodic rebuilds. Specifically, it enforces a relaxed $\alpha$-RNG property to stabilize recall under dynamic workloads. During insertions, it queries the existing graph to identify candidate neighbors and reconstructs local connections by applying the $\alpha$-RNG rules to the newly inserted nodes. During deletions, it repairs the graph by reestablishing connections among affected neighboring nodes. Similar graph repair strategies are also adopted by LSM-VEC [37], OdinANN [36], and DGAI [41], which rebuild or adjust edge connections locally for nodes impacted by insertions and deletions, thereby mitigating quality degradation caused by dynamic updates.

## V. TREE-BASED METHODS

Tree-based methods constitute an important class of disk-resident VSS systems, particularly for low- to medium-dimensional data. As shown in Fig. 10, rather than organizing data via inverted lists or navigating proximity graphs, these approaches rely on explicit tree structures, typically $B^+$-tree–style indexes, to manage disk-resident vectors and control access order. Although they differ in their vector transformations, most tree-based methods follow a shared external-memory design philosophy: vectors are first mapped into orderable and low-dimensional representations and then organized within tree-managed disk blocks to enable efficient pruning and controlled traversal during search. We review their techniques from two aspects: (1) *Block-Oriented Partitioning and Layouts* (Section V-A), and (2) *Block Filtering and Query Execution* (Section V-B).

### A. Block-Oriented Partitioning and Layouts

A central goal of tree-based disk-resident VSS methods is to organize vectors into disk blocks that can be accessed and pruned efficiently under external-memory constraints. Unlike inverted files or graph layouts, which emphasize inverted lists or neighbor links, tree-based methods rely primarily on data ordering to construct hierarchical index structures. This ordering determines how vectors are partitioned into blocks, how blocks are laid out on disk, and ultimately how effectively block access

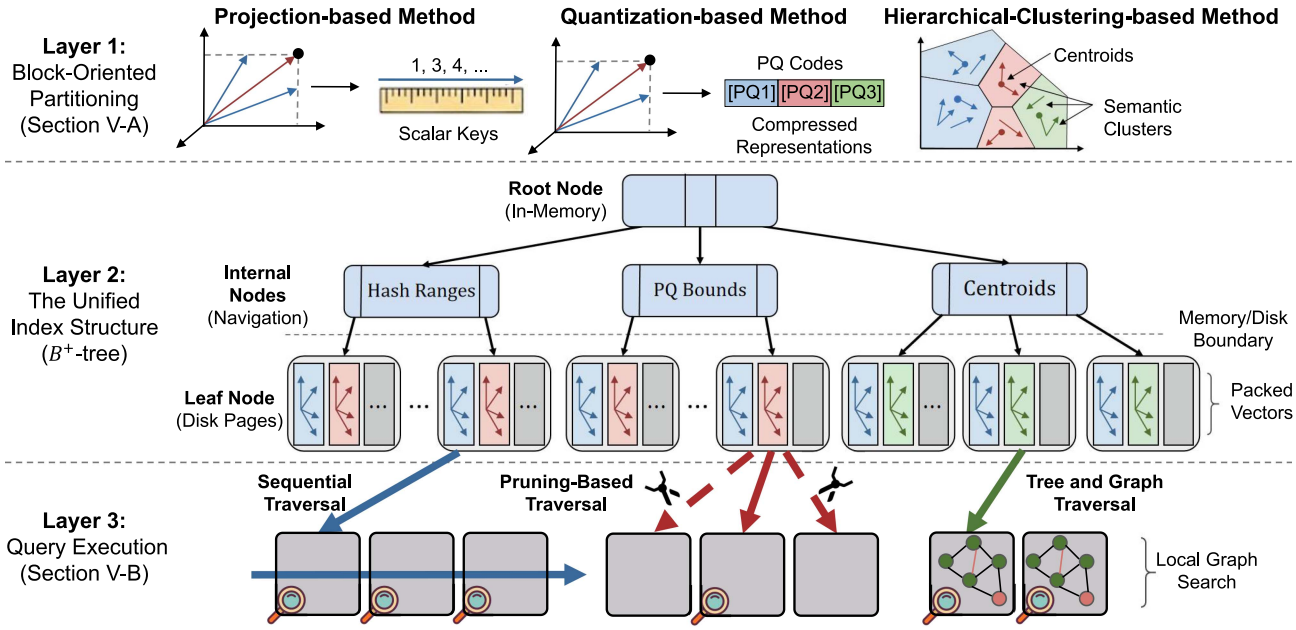



Fig. 10. Illustration of tree-based methods.

can be exploited during search. We classify existing methods into three classes: projection-based, quantization-based, and hierarchical-clustering-based methods.

*Projection-based Methods:* A large class of tree-based methods achieves block-oriented layouts by first mapping high-dimensional vectors into orderable or low-dimensional representations. Early approaches such as C2LSH [91], QALSH [9], SRS [92], I-LSH [45], and EI-LSH [46] employ sets of 2-stable hash functions to generate randomized projections, producing multiple hash values per vector. These hash values are then indexed using tree structures. More recent learning to hash (L2H) based approaches like [47] follow the same structural principle but replace random projections with data-dependent learned mappings.

From a layout perspective, both approaches result in ordered sequences of keys managed by tree leaf nodes, enabling block-level locality and range-based access on disk. Differences between randomized and learned mappings primarily affect the quality of locality within blocks, rather than the underlying partitioning or storage structure.

*Quantization-Based Methods:* PQBF [48] organizes disk-resident data based on PQ partitions. It builds a separate PQB$^+$-tree for each partition, forming a PQB$^+$-forest. Within each PQB$^+$-tree, PQ codes are arranged according to a linear order designed to approximately preserve similarity under asymmetric quantizer distance (AQD), enabling tree-based indexing over compressed representations. PQBF uses B$^+$-trees to manage large collections of PQ codes on disk, achieving compact storage and structured access. However, the imposed linear order does not strictly guarantee disk-page locality, and sequential access during search is not always ensured.

*Hierarchical-Clustering-Based Methods:* B+ANN [49] creates a highly localized disk layout by partitioning the dataset before indexing. It employs recursive $k$-means [60] to group vectors into semantically coherent partitions, ensuring that vectors within each partition are closely related. These final partitions are then used to construct a B$^+$-tree variant where the inner nodes store cluster centroids as keys for navigation, and the leaf nodes store the actual vectors. By physically co-locating the vectors of each leaf node into contiguous disk blocks, this design explicitly maps the semantic hierarchy from clustering directly onto the physical storage, optimizing the layout for efficient, sequential block access.

### B. Block Filtering and Query Execution

Tree-based methods differ primarily in how they filter candidate blocks and execute queries under external-memory constraints. Based on their data partitioning and traversal strategies, we categorize existing approaches into three classes: (1) sequential traversal over projections, (2) pruning-based traversal over quantized codes, and (3) tree and graph traversal over hierarchical structures.

*Sequential Traversal over Projections:* In projection-based methods like C2LSH [91] and QALSH [9], query processing involves expanding search ranges over projected values to retrieve candidate objects from B$^+$-tree indexes. However, these earlier methods adopt a bucket exponential expansion strategy, where the search range (bucket width) is enlarged by a factor of $c$ in each round. This approach often leads to a large number of I/Os, as all points within the newly expanded bucket must be visited to maintain theoretical guarantees, even if a suitable candidate is found early in the round. Furthermore, this fixed expansion can be inefficient, sometimes retrieving too many new objects and other times retrieving none at all.

To address these inefficiencies, methods like I-LSH [45] and EI-LSH [46] adopt an incremental search strategy. Instead of expanding buckets exponentially, they access projected objects

TABLE VI
DATASET SUMMARY

| Dataset | #Dim | #N | Origin | Distance Metric | Query Size | RC | LID | Feature |
|---|---|---|---|---|---|---|---|---|
| MovieLens-10M [93] | 65,134 | 69 K | Set | Jaccard | N/A | 1.66 | 14.4 | ✓ |
| Kosarak [94] | 27,983 | 7 K | Set | Jaccard | N/A | 2.34 | 37.3 | × |
| OpenAI-3072 [95] | 3,072 | 1 M | Text | Euclidean | N/A | 1.94 | 18.9 | ✓ |
| WIT-ResNet50 [96] | 2,048 | 45 K | Image | Euclidean | N/A | 1.45 | 32.6 | × |
| OpenAI-1536 [97] | 1,536 | 1 M | Text | Euclidean | N/A | 2.43 | 15.4 | ✓ |
| Enron [98] | 1,369 | 94 K | Text | Angular | 200 | 6.42 | 13.6 | ✓ |
| Wiki [99] | 1,024 | 247.2 M | Text | Angular | N/A | 2.38 | 14.2 | ✓ |
| MSMARCO [100] | 1,024 | 53.2 M | Text | Angular | 1,677 | 1.69 | 22.6 | ✓ |
| GIST [11] | 960 | 1 M | Image | Euclidean | 1,000 | 1.94 | 18.9 | × |
| MNIST [101] | 784 | 60 K | Image | Euclidean | 10,000 | 2.43 | 12.7 | ✓ |
| LAION-5B [102] | 768 | 5B | Image | Angular | N/A | 2.27 | 14.2 | ✓ |
| COCO-I2I [103] | 512 | 113K | Image | Angular | 10,000 | 3.32 | 9.37 | × |
| MSong [104] | 420 | 1 M | Audio | Euclidean | 200 | 3.81 | 9.50 | × |
| Tiny [105] | 384 | 5 M | Image | Euclidean | 10,000 | 1.77 | 32.8 | × |
| GloVe-300 [106] | 300 | 2.2 M | Text | Angular | 10,000 | 1.96 | 28.1 | × |
| Crawl [107] | 300 | 2 M | Text | Angular | 10,000 | 2.44 | 15.9 | × |
| Facebook SimSearchNet++ [108] | 256 | 1B | Image | Euclidean | 100,000 | 1.20 | 70.0 | × |
| UQvideo [109] | 256 | 1 M | Video | Euclidean | 10,000 | 8.39 | 7.20 | × |
| NYTimes [110] | 256 | 290 K | Text | Angular | 10,000 | 1.82 | 19.3 | × |
| Yandex Text-to-Image [108] | 200 | 1B | Text & Image | Inner-product | 100,000 | 3.94 | 4.44 | × |
| GloVe-200 [106] | 200 | 1.2 M | Text | Angular | 10,000 | 2.13 | 26.3 | × |
| Audio [111] | 192 | 53 K | Audio | Euclidean | 200 | 2.96 | 16.1 | × |
| SIFT [112] | 128 | 1B | Image | Euclidean | 10,000 | 3.50 | 9.30 | × |
| SPACEV [108] | 100 | 1.4B | Text | Euclidean | 29,316 | 2.24 | 17.4 | × |
| Turing [108] | 100 | 1B | Text | Euclidean | 100,000 | 1.56 | 32.6 | × |
| GloVe-100 [106] | 100 | 1.2 M | Text | Angular | 10,000 | 1.82 | 20.0 | × |
| DEEP [113] | 96 | 1B | Image | Euclidean | 10,000 | 1.96 | 12.1 | × |

sequentially in increasing order of their distance to the query on the projected dimension. This allows for a more fine-grained search that prioritizes the most promising candidates first, significantly increasing the proportion of sequential disk accesses. This incremental approach also enables the use of a more aggressive early termination condition, which stops the search as soon as a candidate is found with a high probability of being the correct answer, thus reducing I/O costs while still maintaining theoretical guarantees.

Li et al. [47] follow the same execution paradigm but benefit from higher-quality ordering. Since learned projections are optimized to preserve neighborhood structure, blocks accessed consecutively during range scans are more likely to contain relevant candidates. As a result, learned methods typically reduce the number of scanned blocks compared to random projections. Nevertheless, when multiple projections or lists are involved, random I/O remains unavoidable during candidate aggregation and verification.

*Pruning-Based Traversal over Quantization Codes:* The quantization-based methods, such as PQBF [48], rely less on long sequential scans and more on tree-based pruning. Queries traverse selected PQB$^+$-trees using AQD lower bounds to eliminate irrelevant subtrees. While this significantly reduces the number of accessed blocks, the imposed linear order over PQ codes does not guarantee disk-page adjacency. Consequently, PQBF's query execution often involves non-trivial random I/O, with efficiency gains stemming primarily from pruning rather than sequential access.

*Tree and Graph Traversal over Hierarchical Structures:* B+ANN [49] implements a hybrid query execution strategy that minimizes disk I/O by separating coarse-grained filtering from fine-grained search. During a query, a traversal begins from the root of the B+ANN tree, using only the inner nodes, which can be kept in memory, to navigate the index. This in-memory traversal acts as an efficient filter, quickly identifying a small set of promising leaf-node blocks on disk without reading the vectors themselves. Disk access is then confined to loading only these targeted blocks. Once in memory, a more precise, graph-like greedy search can be performed using pre-built "skip-edge" connections between vectors within these leaf nodes to refine the final candidates. This two-phase execution model converts queries into a small number of coarse-grained, largely sequential block reads and performs graph-like refinement within each loaded block, substantially reducing the impact of random I/O latency.

## VI. BENCHMARKS AND PRACTICAL GUIDELINES

This section summarizes commonly used datasets, presents a unified evaluation framework, and provides practical guidelines for selecting disk-resident VSS methods.

### A. Datasets and Evaluation Framework

*Datasets:* Table VI summarizes the key characteristics of representative datasets, including dimensionality, dataset size, data origin, distance metric, query workload size (when available), and whether feature attributes are provided. The table also includes two dataset difficulty metrics, namely Relative Contrast (RC) and Local Intrinsic Dimensionality (LID). All datasets and the difficulty evaluation tools are publicly available

in our GitHub repository,[1] which also provides brief dataset descriptions. Below, we introduce the two difficulty metrics and report their evaluation results.

1) *Relative Contrast (RC):* RC measures the separability of true nearest neighbors by comparing the typical query-to-database distance with the nearest-neighbor distance. An RC value closer to 1 indicates strong distance concentration and, consequently, a more challenging search problem, whereas larger RC values imply clearer distance separation and therefore an easier search regime. Using our developed tool, we compute the RC values for all datasets listed in Table VI. The results show that Facebook SimSearchNet++ exhibits RC values closest to 1, reflecting severe global distance concentration. In contrast, datasets such as UQvideo exhibit substantially larger RC values, indicating clearer global distance separation.

2) *Local Intrinsic Dimensionality (LID):* LID characterizes the *local growth rate* of the neighborhood around a query, capturing how rapidly the number of nearby points increases as the radius expands. In practice, LID is estimated from the empirical distances to a query's nearest neighbors. Higher LID values indicate that many points lie at similar distances within the local neighborhood, making nearest-neighbor identification more ambiguous and thus more difficult. Lower LID values suggest stronger local contrast and typically an easier search regime. We evaluate the LID values for all datasets, and the results show that Facebook SimSearchNet++ exhibits particularly high LID values, suggesting complex and rapidly expanding local neighborhoods. In contrast, datasets such as UQvideo exhibit much lower LID values, reflecting simpler local geometry and well-separated neighborhood structures.

*Unified Evaluation Framework:* Beyond standardized datasets, fair cross-paper comparison also requires consistent evaluation metrics and experimental settings. We therefore provide a unified evaluation framework for disk-resident VSS, covering query accuracy and efficiency, disk I/O, computation, storage, caching, updates, construction, scalability, and parameter sensitivity. In particular, we recommend reporting I/O utilization, normalized memory and disk footprints, and sensitivity to key parameters such as page size and beam width. Hardware configurations and major algorithmic parameters should also be explicitly reported to improve reproducibility. Detailed metrics and recommended reporting guidelines are provided in Appendix A, available online (Evaluation Framework).

Based on these common metrics, we further provide a qualitative comparison of major disk-resident index paradigms in Appendix B, available online (Comparison under Common Evaluation Metrics), covering query latency, QPS, Recall@$k$, I/O cost, memory usage, build time, and update efficiency. This comparison highlights their typical system-level trade-offs rather than absolute performance under a unified experimental setting.

[1]https://github.com/hkbudb/disk-vss-survey

TABLE VII
RECOMMENDATION OF VSS METHODS IN DIFFERENT SCENARIOS

| Scenario | Recommended Methods |
|---|---|
| Prioritize memory efficiency | AiSAQ, LM-DiskANN |
| High-throughput ANN search | PipeANN |
| Frequently updated ANN search | SPFresh, FreshDiskANN, DGAI |
| Very high-dimensional vectors | AlayaLaser |
| Page-oriented search | Starling, PageANN |
| GPU-equipped platforms | FusionANNS, FlashANNS |
| On-device deployment | MicroNN |
| Filtered ANN workloads | Hybrid IVFFlat, Filtered-DiskANN |
| Range search | DiskJoin, Starling |
| Vector similarity join | DiskJoin |

### B. Practical Selection Guide

In this section, we provide a practical selection guide for disk-resident VSS methods based on query performance requirements, query types, update frequency, memory budget, and available hardware resources. As summarized in Table VII, for high-throughput ANN workloads, PipeANN is a suitable choice because it overlaps computation with asynchronous I/O. When memory capacity is the primary constraint, AiSAQ and LM-DiskANN reduce the DRAM footprint through all-in-disk or memory-efficient index designs. Dynamic workloads require different solutions depending on the underlying index structure. IVF-based method like SPFresh supporting incremental maintenance is suitable for this scenario, whereas FreshDiskANN and DGAI are designed for frequently updated graph-based indexes. For more specialized environments, AlayaLaser targets very high-dimensional vectors; Starling and PageANN improve page-oriented graph traversal; FusionANNS and FlashANNS exploit GPU-equipped heterogeneous platforms; MicroNN is tailored to resource-constrained on-device deployment. Query semantics also affect method selection. Hybrid IVFFlat and Filtered-DiskANN are designed for filtered ANN workloads, Starling supports approximate range search, and DiskJoin targets vector similarity joins. Note that these recommendations reflect the primary design objectives of the methods, rather than providing a universal performance ranking, as their relative effectiveness may vary depending on the dataset, accuracy target, hardware configuration, and workload characteristics.

## VII. FURTHER DIRECTIONS

Despite significant progress, disk-resident VSS remains a rapidly evolving research area with many open challenges. Below, we outline several promising directions that are not yet fully addressed by existing methods, but are becoming increasingly critical as vector databases move toward larger-scale deployments, support more mixed and dynamic workloads, and operate in cloud-native, multi-tenant, and privacy-constrained environments.

*1) Unified Support for Mixed and Dynamic Workloads:* While most existing disk-resident VSS methods focus on optimizing a single aspect, such as ANN queries, FANN queries, or update efficiency, real-world deployments often require *joint support* for mixed query types under dynamic workloads. In practice, a single system may need to simultaneously serve

high-recall ANN queries, FANN queries, and continuous insertions or deletions, each exhibiting distinct access patterns, pruning behaviors, and I/O characteristics. Naively supporting such mixed workloads using separate indexes or independently tuned execution paths can lead to excessive storage overhead, duplicated I/O, and suboptimal cache utilization. Future research could explore unified index structures and adaptive frameworks that dynamically balance ANN and FANN processing while efficiently handling updates.

*2) Exploiting Emerging Storage Hardware and Cloud-Native Environments:* Most disk-resident VSS techniques are originally designed for local SSDs with relatively stable latency and bandwidth characteristics. However, modern large-scale deployments increasingly rely on cloud-native storage infrastructures [114], [115], [116], [117], [118], including disaggregated storage, remote NVMe, and object stores, where access latency, throughput, and cost can vary significantly across storage tiers. These environments challenge traditional assumptions regarding block size, sequential access, and caching behavior, and often expose new trade-offs between performance elasticity and cost efficiency. Future research may investigate storage-aware and cloud-native VSS designs that explicitly account for heterogeneous storage properties, such as tier-aware indexing, adaptive block layouts, and elastic query execution strategies that migrate or replicate index components across memory, SSDs, and object storage. Co-designing disk-resident VSS with cloud orchestration mechanisms and resource schedulers also presents an important direction for enabling scalable and cost-efficient vector similarity search.

*3) Privacy-Preserving and Federated Disk-Resident VSS:* As vector databases are increasingly deployed in privacy-sensitive, multi-tenant, and cross-organizational environments, supporting disk-resident VSS under strict data isolation and privacy constraints becomes an important open challenge. In such settings, vectors are often distributed across multiple sites and cannot be centrally aggregated, necessitating federated vector similarity search with limited information sharing, constrained communication budgets, and potentially untrusted execution environments. Existing VSS methods rarely consider the interaction between disk-efficient indexing, communication overhead, and privacy guarantees. Future research could explore federated and privacy-preserving disk-resident VSS designs that integrate secure indexing, encrypted or compressed representations, and communication-aware query processing. Developing pruning and early-termination techniques that reduce cross-site communication while preserving search accuracy is particularly important for enabling trustworthy vector search in federated and privacy-constrained deployments.

*4) Parameter Tuning and Auto-Configuration:* Disk-resident VSS systems typically expose a large number of parameters, including index configurations, block sizes, memory budgets, pruning thresholds, and cache sizes, all of which critically influence query latency, accuracy, and I/O efficiency. In practice, these parameters are often manually tuned based on empirical heuristics or assumed workload characteristics, making them difficult to generalize across datasets, hardware platforms, and deployment environments. Moreover, optimal configurations may change over time as workloads evolve or system resources fluctuate. Developing systematic parameter tuning and auto-configuration mechanisms therefore remains a significant challenge. Promising directions include workload-aware and data-aware tuning strategies, online performance monitoring with adaptive reconfiguration, and learning-based approaches that automatically balance accuracy, latency, and I/O costs under dynamic conditions.

*5) Disk-Resident VSS as a System Substrate for RAG and LLM-Centric Applications:* Retrieval-Augmented Generation (RAG) and large language model (LLM)–centric applications increasingly rely on vector databases as an external memory to store and retrieve long-term knowledge, conversational history, and task-specific context. Unlike traditional ANN workloads, these applications introduce new access patterns and system requirements, including multi-granularity retrieval (e.g., passages, documents, episodic memories), strict latency constraints in interactive settings, and frequent memory updates driven by continual learning or user interactions. Moreover, LLM memory workloads often require semantic freshness, temporal locality, and selective forgetting, which are not well supported by existing disk-resident VSS designs. Future research could investigate disk-resident VSS architectures that explicitly model LLM memory semantics, such as time-aware or session-aware indexing and query processing mechanisms jointly optimized for retrieval quality and generation efficiency. Bridging disk-resident VSS with RAG pipelines also raises new challenges in end-to-end optimization, including co-designing retrieval granularity, caching policies, and pruning strategies to better align vector search behavior with downstream generation objectives.

*6) Standardized Benchmarking and Reproducibility:* Evaluating disk-resident VSS systems remains challenging due to the diversity of storage architectures, index layouts, caching strategies, and workload assumptions adopted across existing studies. Therefore, prior work often relies on different datasets, metrics, and experimental configurations, complicating fair and reproducible comparison. Establishing standardized benchmarks and evaluation protocols that capture realistic storage hierarchies, hybrid workloads, and cost–performance trade-offs is therefore essential. Open-source implementations, unified experimental frameworks, and reproducible end-to-end evaluation pipelines would greatly facilitate systematic comparison and accelerate progress in disk-resident VSS research.

## VIII. Conclusion

In this survey, we have presented a comprehensive and structured review of disk-resident VSS. We categorize existing methods into IVF-based, graph-based, and tree-based approaches, and systematically analyze their key design components, including index construction, block-aware storage and layouts, query execution, and update and maintenance mechanisms. Beyond method classification, we summarize commonly used benchmark datasets, establish a unified evaluation framework for fair and reproducible comparison. We further provide practical guidelines for selecting suitable methods under different workload, resource, and hardware constraints. Finally, we identify key

open challenges and promising research directions for future disk-resident VSS systems. We hope this survey provides a useful reference for understanding the design space, evaluating existing methods, and guiding future research and practical system development.

## REFERENCES


[1] C. Wei et al., "Analyticdb-V: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3152–3165, 2020.

[2] B. Agüera y Arcas et al., "Now playing: Continuous low-power music recognition," 2017, *arXiv:1711.10958* .

[3] Y. Rui, T. S. Huang, and S.-F. Chang, "Image retrieval: Current techniques, promising directions, and open issues," *J. Vis. Commun. Image Representation*, vol. 10, no. 1, pp. 39–62, 1999.

[4] P. Gao et al., "Trae agent: An LLM-based agent for software engineering with test-time scaling," 2025, *arXiv:2507.23370*.

[5] M. Arslan, H. Ghanem, S. Munawar, and C. Cruz, "A survey on RAG with LLMs," *Procedia Comput. Sci.*, vol. 246, pp. 3781–3790, 2024.

[6] W. Fan et al., "A survey on rag meeting LLMs: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 6491–6501.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 9112–9141.

[8] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 11, pp. 2227–2240, Nov. 2014.

[9] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, "Query-aware locality-sensitive hashing for approximate nearest neighbor search," *Proc. VLDB Endowment*, vol. 9, no. 1, pp. 1–12, 2015 .

[10] J. Li et al., "A general and efficient querying method for learning to hash," *Proc. Int. Conf. Manage. Data*, 2018, pp. 1333–1347.

[11] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011.

[12] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization for approximate nearest neighbor search," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 2946–2953.

[13] J. Gao and C. Long, "RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search," *Proc. ACM Manage. Data*, vol. 2, no. 3, pp. 1–27, 2024.

[14] F. André, A.-M. Kermarrec, and N. Le Scouarnec, "Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan," *Proc. VLDB Endowment*, vol. 9, no. 4, 2016, Art. no. 12.

[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[16] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proc. VLDB Endowment*, vol. 12, no. 5, pp. 416–474, 2019.

[17] Y. Peng, B. Choi, T. N. Chan, J. Yang, and J. Xu, "Efficient approximate nearest neighbor search in multi-dimensional databases," *ACM SIGMOD*, vol. 1, no. 1, pp. 1–27, 2023.

[18] S. J. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 13766–13776 .

[19] Q. Chen et al., "SPANN:Highly-efficient billion-scale approximate nearest neighborhood search," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 5199–5212.

[20] M. Wang et al., "Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment," *ACM SIGMOD*, vol. 2, no. 1, pp. 1–27, 2024.

[21] Y. Xu et al., "SPFresh: Incremental in-place update for billion-scale vector search," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 545–561.

[22] H. Li and J. Xu, "BAMG: A block-aware monotonic graph index for disk-based approximate nearest neighbor search," in *Proc. IEEE 42nd Int. Conf. Data Eng.*, 2026, pp. 3433–3445.

[23] Z. Yue et al., "Select edges wisely: Monotonic path aware graph layout optimization for disk-based ann search," *Proc. VLDB Endowment*, vol. 18, no. 11, pp. 4337–4349, 2025.

[24] C. Zhang, J. Wang, W.-L. Zhao, and S. Xiao, "Highly efficient disk-based nearest neighbor search on extended neighborhood graph," in *Proc. 48th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2025, pp. 2513–2523.

[25] D. Kang, D. Jiang, H. Yang, H. Liu, and B. Li, "Scalable disk-based approximate nearest neighbor search with page-aligned graph," 2025, *arXiv:2509.25487*.

[26] P. Yin et al., "Gorgeous: Revisiting the data layout for disk-resident high-dimensional vector search," in *Proc. ACM Manage. Data*, vol. 4, no. 3, 2026, pp. 192:1–192:29.

[27] Y. Song et al., "TRIM: Accelerating high-dimensional vector similarity search with enhanced triangle-inequality-based pruning," *Proc. ACM Manage. Data*, vol. 3, 2025, Art. no. 373.

[28] Y. Zhou et al., "GoVector: An I/O-efficient caching strategy for high-dimensional vector nearest neighbor search," *Ruan Jian Xue Bao J. Softw.*, vol. 37, no. 3, pp. 1021–1036, 2026.

[29] J. Pound et al., "MicroNN: An on-device disk-resident updatable vector database," in *Proc. Companion Int. Conf. Manage. Data*, 2025, pp. 608–621.

[30] M. Zhang and Y. He, "Zoom: SSD-based vector search for optimizing accuracy, latency and memory," 2018, *arXiv:1809.04067*.

[31] S. Emanuilov and A. Dimov, "Billion-scale similarity search using a hybrid indexing approach with advanced filtering," *Cybern. Inf. Technol.*, vol. 24, no. 4, pp. 45–58, 2024.

[32] B. Tian et al., "FusionANNs: An efficient CPU/GPU cooperative processing architecture for billion-scale approximate nearest neighbor search," 2024, *arXiv:2409.16576*.

[33] J. Ni et al., "DiskANN : Efficient page-based search over isomorphic mapped graph index using query-sensitivity entry vertex," 2023, *arXiv:2310.00402*.

[34] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshdiskANN: A fast and accurate graph-based ANN index for streaming similarity search," 2021, *arXiv:2105.09613*.

[35] S. Gollapudi et al., "Filtered-diskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf.*, 2023, pp. 3406–3416, 2023.

[36] H. Guo and Y. Lu, "OdinANN: Direct insert for consistently stable performance in billion-scale graph-based vector search," in *Proc. USENIX Conf. File Storage Technol.*, 2026, pp. 133–147.

[37] S. Zhong, D. Mo, and S. Luo, "LSM-VEC: A large-scale disk-based system for dynamic vector search," 2025, *arXiv:2505.17152*.

[38] Y. Chen, X. Yan, A. Meliou, and E. Lo, "DiskJoin: Large-scale vector similarity join with SSD," *Proc. ACM Manage. Data*, vol. 3, no. 6, pp. 1–27, 2025.

[39] N. A. Dang, B. Landrum, and K. Birman, "Passing the baton: High throughput distributed disk-based vector search with batANN," 2025, *arXiv:2512.09331*.

[40] H. Guo and Y. Lu, "Achieving low-latency graph-based vector search via aligning best-first search algorithm with SSD," in *Proc. 19th USENIX Symp. Operating Syst. Des. Implementation*, 2025, pp. 171–186.

[41] J. Lou, Q. Yu, S. Gong, S. Yu, Y. Zhang, and G. Yu, "DGAI: Decoupled on-disk graph-based ANN index for efficient updates and queries," 2025, *arXiv:2510.25401*.

[42] W. Zhao et al., "Optimizing SSD-resident graph indexing for high-throughput vector search," 2026, *arXiv:2602.22805*.

[43] W. Chen et al., "AlayaLaser: Efficient index layout and search strategy for large-scale high-dimensional vector similarity search," 2026, *arXiv:2602.23342*.

[44] Y. Xiao et al., "Breaking the storage-compute bottleneck in billion-scale ANNs: A GPU-driven asynchronous I/O framework," 2025, *arXiv:2507.10070*.

[45] W. Liu, H. Wang, Y. Zhang, W. Wang, and L. Qin, "I-lsh: I/I efficient c-approximate nearest neighbor search in high-dimensional space," in *Proc. Int. Conf. Data Eng.*, 2019, pp. 1670–1673.

[46] W. Liu, H. Wang, Y. Zhang, W. Wang, L. Qin, and X. Lin, "EI-LSH: An early-termination driven i/o efficient incremental C-approximate nearest neighbor search," *VLDBJ*, vol. 30, pp. 215–235, 2021.

[47] M. Li, Y. Zhang, Y. Sun, W. Wang, I. W. Tsang, and X. Lin, "I/O efficient approximate nearest neighbour search based on learned functions," in *Proc. Int. Conf. Data Eng.*, 2020, pp. 289–300 .

[48] Y. Liu, H. Cheng, and J. Cui, "PQBF: I/O-efficient approximate nearest neighbor search by product quantization," in *Proc. Conf. Inf. Knowl. Manage.*, 2017, pp. 667–676.

[49] S. F. Tekin and R. Bordawekar, "B ANN: A fast billion-scale disk-based nearest-neighbor index," 2025, *arXiv:2511.15557*.

[50] Z. Zhang, F. Liu, G. Huang, X. Liu, and X. Jin, "Fast vector query processing for large datasets beyond GPU memory with reordered pipelining," in *Proc. 21st USENIX Symp. Networked Syst. Des. Implementation*, 2024, pp. 23–40.

[51] W. Li et al., "Approximate nearest neighbor search on high dimensional data—Experiments, analyses, and improvement," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, Aug. 2020.

[52] Y. Tian, Z. Yue, R. Zhang, X. Zhao, B. Zheng, and X. Zhou, "Approximate nearest neighbor search in high dimensional vector databases: Current research and future directions," *IEEE Data Eng. Bull.*, vol. 47, no. 3, pp. 39–54, Mar. 2023.

[53] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," in *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 1964–1978, 2021.

[54] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," 2025, *arXiv:2505.06501*.

[55] J. Gao et al., "High-dimensional vector quantization: General framework, recent advances, and future directions," *IEEE Data Eng. Bull.*, vol. 48, no. 3, pp. 3–19, Mar. 2024.

[56] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDBJ*, vol. 33, no. 5, pp. 1591–1615, 2024.

[57] X. Chen et al., "Disk-resident graph ann search: An experimental evaluation," 2026, *arXiv:2603.01779*.

[58] L. Li, S. Gong, Y. Yang, Y. Wang, and J. Wu, "I/O optimizations for graph-based disk-resident approximate nearest neighbor search: A design space exploration," in *Proc. VLDB Endowment*, vol. 19, no. 7, pp. 1484–1498, 2026.

[59] K. Lu, M. Kudo, C. Xiao, and Y. Ishikawa, "HVS: Hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search," *Proc. VLDB Endowment*, vol. 15, no. 2, pp. 246–258, 2021.

[60] K. Krishna and M. N. Murty, "Genetic k-means algorithm," *IEEE Trans. Syst., Man, Cybern., Part B (Cybern.)*, vol. 29, no. 3, pp. 433–439, Jun. 1999.

[61] M. I. Malinen and P. Fränti, "Balanced k-means for clustering," in *Proc. Joint IAPR Int. Workshops Stat. Techn. Pattern Recognit. Struct. Syntactic Pattern Recognit.*, 2014, pp. 32–41.

[62] P. S. Bradley, K. P. Bennett, and A. Demiriz, "Constrained K-means clustering," *Microsoft Res.*, Redmond, 2000, pp. 1–8.

[63] H. Liu, Z. Huang, Q. Chen, M. Li, Y. Fu, and L. Zhang, "Fast clustering with flexible balance constraints," in *Proc. IEEE Int. Conf. Big Data*, 2018, pp. 743–750.

[64] J. Mohoney et al., "Quake: Adaptive indexing for vector search," in *Proc. 19th USENIX Symp. Operating Syst. Des. Implementation*, Jul. 2025, pp. 153–169.

[65] R. Guo et al., "Accelerating large-scale inference with anisotropic vector quantization," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3887–3896.

[66] C. Aguerrebere, I. Bhati, M. Hildebrand, M. Tepper, and T. Willke, "Similarity search in the blink of an eye with compressed indices," in *Proc. VLDB Endowment*, vol. 16, no. 11, 2023, pp. 3433–3446.

[67] C. Aguerrebere, M. Hildebrand, I. S. Bhati, T. Willke, and M. Tepper, "Locally-adaptive quantization for streaming vector search," 2024, *arXiv:2402.02044*.

[68] M. AI, "Faiss," 2017. [Online]. Available: https://ai.facebook.com/tools/faiss

[69] Y. Gou, J. Gao, Y. Xu, and C. Long, "SymphonyQG: Towards symphonious integration of quantization and graph for approximate nearest neighbor search," *ACM SIGMOD*, vol. 3, no. 1, pp. 1–26, 2025.

[70] X. Zhong et al., "VSAG: An optimized search framework for graph-based approximate nearest neighbor search," *Proc. VLDB Endowment*, vol. 18, no. 12, 2025, pp. 5017–5030.

[71] A. X. Liu, K. Shen, and E. Torng, "Large scale hamming distance query processing," in *Proc. Int. Conf. Data Eng.*, 2011, pp. 553–564.

[72] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. TIT-28, no. 2, pp. 129–137, Mar. 1982.

[73] D. Sculley, "Web-scale K-means clustering," *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 1177–1178.

[74] C. Huan et al., "OrchANN: A unified I/O orchestration framework for skewed out-of-core vector search," 2025, *arXiv:2512.22838*.

[75] B. Bratić, M. E. Houle, V. Kurbalija, V. Oria, and M. Radovanović, "NN-descent on high-dimensional data," in *Proc. 8th Int. Conf. Web Intell. Mining Semantics*, 2018, pp. 1–8.

[76] K. Tatsuno, D. Miyashita, T. Ikeda, K. Ishiyama, K. Sumiyoshi, and J. Deguchi, "AiSAQ: All-in-storage ANNs with product quantization for dram-free information retrieval," 2024, *arXiv:2404.06004*.

[77] Y. Pan, J. Sun, and H. Yu, "LM-diskANN: Low memory footprint in disk-native dynamic graph-based ann indexing," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 5987–5996.

[78] Y. Wang et al., "LEANN: A low-storage vector index," 2025, *arXiv:2506.08276*.

[79] H. Liu, B. Tian, Z. Duan, X. Liao, and Y. Zhang, "A smartSSD-based near data processing architecture for scalable billion-point approximate nearest neighbor search," *ACM Trans. Storage*, vol. 22, 2025, Art. no. 23.

[80] J. Ren, M. Zhang, and D. Li, "HM-ANN: Efficient billion-point nearest neighbor search on heterogeneous memory," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 10672–10684.

[81] J. Sun et al., "GaussDB-vector: A large-scale persistent real-time vector database for LLM applications," *Proc. VLDB Endowment*, vol. 18, no. 12, pp. 4951–4963, 2025.

[82] Z. Song, Wang, Bin, Yang, Xiaochun, and J. Zhang, "Low-latency out-of-core ann search in high-dimensional space," 2026, *arXiv:2605.05787*.

[83] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Syst. J.*, vol. 5, no. 2, pp. 78–101, 1966.

[84] R. Guo et al., "Manu: A cloud native vector database management system," *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3548–3561, 2022.

[85] J. Wang et al., "Milvus: A purpose-built vector data management system," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 2614–2627.

[86] H. Jiang, H. Guo, M. Xie, J. Shu, and Y. Lu, "High-throughput, cost-effective billion-scale vector search with a single GPU," *Proc. ACM Manage. Data*, vol. 3, no. 6, 2025, Art. no. 334.

[87] K. Venkatasubba, S. Khan, S. Singh, H. V. Simhadri, and J. Vedurada, "BANG: Billion-scale approximate nearest neighbour search using a single GPU," *IEEE Trans. Big Data*, vol. 11, no. 06, pp. 3142–3157, Dec. 2025.

[88] Y. Peng et al., "SVFusion: A CPU-GPU co-processing architecture for large-scale real-time vector search," in *Proc. VLDB Endowment*, vol. 19, no. 5, 2026, pp. 1074–1087.

[89] M. Wang, H. Wu, X. Ke, Y. Gao, Y. Zhu, and W. Zhou, "Accelerating graph indexing for ANNs on modern CPUs," *ACM SIGMOD*, vol. 3, no. 3, 2025, Art. no. 123.

[90] Z. Song, B. Wang, and X. Yang, "Accelerating high-dimensional ANN search via skipping redundant distance computations," *ACM SIGMOD*, vol. 3, no. 6, 2025, Art. no. 288.

[91] J. Gan, J. Feng, Q. Fang, and W. Ng, "Locality-sensitive hashing scheme based on dynamic collision counting," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2012, pp. 541–552.

[92] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin, "SRS: Solving C-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index," *Proc. VLDB Endowment*, vol. 8, pp. 1–12, 2014.

[93] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Trans. Interactive Intell. Syst.*, vol. 5, no. 4, pp. 1–19, 2015.

[94] B. Goethals and M. J. Zaki, "Advances in frequent itemset mining implementations: Report on fimi'03," *ACM SIGKDD Explorations Newslett.*, vol. 6, no. 1, pp. 109–117, 2004.

[95] Qdrant, "dbpedia-entities-openai3-text-embedding-3-large-3072-1m," Hugging Face Datasets, Feb. 2024. [Online]. Available: https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M

[96] K. Srinivasan, K. Raman, J. Chen, M. Bendersky, and M. Najork, "WIT: Wikipedia-based image text dataset for multimodal multilingual machine learning," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 2443–2449.

[97] Qdrant, "dbpedia-entities-openai3-text-embedding-3-large-1536-1m" Hugging Face Datasets, Feb. 2024. [Online]. Available: https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M

[98] B. Klimt and Y. Yang, "The enron corpus: A new dataset for email classification research," in *Proc. Eur. Conf. Mach. Learn.*, 2004, pp. 217–226.

[99] W. Foundation, "Wikimedia downloads," (n.d.). Accessed: January 22, 2026. [Online]. Available: https://dumps.wikimedia.org

[100] T. Nguyen et al., "MS MARCO: A human-generated machine reading comprehension dataset," in *Proc. Workshop Cognit. Comput.: Integrating Neural Symbolic Approaches*, 2016.

[101] Y. LeCun, C. Cortes, and C. Burges, "Mnist handwritten digit database," ATT Labs, 2010. [Online]. Available: http://yann.lecun.com/exdb/mnist

[102] C. Schuhmann et al., "Laion-5B: An open large-scale dataset for training next generation image-text models," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2022, pp. 25278–25294.

[103] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
[104] T. Bertin-Mahieux, D. P. Ellis, B. Whitman, and P. Lamere, "The million song dataset," in *Proc. Proc. 12th Int. Soc. Music Inf. Retrieval Conf.*, 2011, pp. 591–596.
[105] A. Torralba, R. Fergus, and W. T. Freeman, "80 million tiny images: A large data set for nonparametric object and scene recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 30, no. 11, pp. 1958–1970, Nov. 2008.
[106] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 1532–1543. [Online]. Available: http://www.aclweb.org/anthology/D14-1162
[107] T. Mikolov, E. Grave, P. Bojanowski, C. Puhrsch, and A. Joulin, "Advances in pre-training distributed word representations," in *Proc. 11th Int. Conf. Lang. Resour. Eval.*, 2018, pp. 52–55.
[108] H. V. Simhadri et al., "Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 177–189.
[109] J. Song, Y. Yang, Z. Huang, H. T. Shen, and R. Hong, "Multiple feature hashing for real-time large scale near-duplicate video retrieval," in *Proc. 19th ACM Int. Conf. Multimedia*, 2011, pp. 423–432.
[110] D. Newman, "Bag of words," *UCI Mach. Learn. Repository*, 2008, doi: 10.24432/C5ZG6P.
[111] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proc. 20th Int. Conf. World Wide Web*, 2011, pp. 577–586.
[112] H. Jégou, R. Tavenard, M. Douze, and L. Amsaleg, "Searching in one billion vectors: Re-rank with source coding," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2011, pp. 861–864.
[113] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2055–2063.
[114] C. Huang, "Migrating from S3 vectors to zilliz cloud: Unlocking the power of tiered storage," (n.d.). 2025. [Online]. Available: https://zilliz.com/blog/migrating-from-s3-vectors-to-zilliz-cloud-unlocking-the-power-of-tiered-storage
[115] C. Yun, "Introducing amazon S3 vectors: First cloud storage with native vector support at scale (preview)," (n.d.). 2025. [Online]. Available: https://aws.amazon.com/cn/blogs/aws/introducing-amazon-s3-vectors-first-cloud-storage-with-native-vector-support-at-scale/
[116] T. Inc, "Turbopuffer — serverless vector & full-text search built from first principles on object storage," Company Website / Product page, (n.d.). Accessed: January 22, 2026. [Online]. Available: https://turbopuffer.com/
[117] Y. Song, X. Zhou, C. S. Jensen, and J. Xu, "Vector search for the future: From memory-resident, static heterogeneous storage, to cloud-native architectures," in *Proc. Companion Int. Conf. Manage. Data*, 2026, pp. 580–587.
[118] Z. Li et al., "Cloud-native vector search: A comprehensive performance analysis," 2025, *arXiv:2511.14748*.



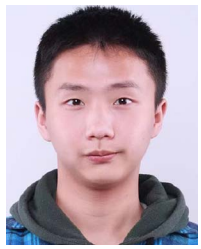
**Yitong Song** received the BEng degree in computer science from the China University of Geosciences and the PhD degree in computer science from Shanghai Jiao Tong University. She is currently a postdoctoral fellow with the Department of Computer Science, Hong Kong Baptist University. Her research interests include vector databases and AI data infrastructure.



**Huiling Li** received the BEng degree in software engineering from Zhengzhou University in 2021. He is currently working toward the PhD degree with the Department of Computer Science, Hong Kong Baptist University. His research interests include multi-agent computing, vector databases, and spatiotemporal data processing.



**Zheng Wu** received the BEng degree in software engineering from Zhejiang Normal University in 2024. He is currently working toward the PhD degree with the Department of Computer Science, Hong Kong Baptist University. His research interests include graph algorithms and vector data management.



**Lanjing Yi** received the BEng degree in computer science and engineering from the Southern University of Science and Technology in 2023. He is currently working toward the PhD degree with the Department of Computer Science, Hong Kong Baptist University. His research interests include efficient inference of large language models and long-term memory.



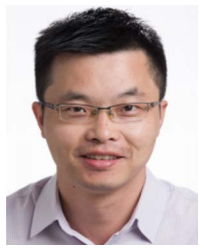
**Bojian Zhu** received the BEng degree in computer science and technology from Xidian University in 2024. He is currently working toward the PhD degree with the Department of Computer Science, Hong Kong Baptist University. His research interests include spatiotemporal data management, parallel computing, and vector databases.



**Xuanhe Zhou** received the PhD degree in computer science from Tsinghua University. He is currently an assistant professor wit the Department of Computer Science, Shanghai Jiao Tong University. His research focuses on data systems. He was the recipient of the SIGMOD 2025 Jim Gray Dissertation Honorable Mention and VLDB 2023 Best Industry Paper Runner-up.



**Xin Huang** received the PhD degree from the Chinese University of Hong Kong in 2014. He is currently an associate professor with Hong Kong Baptist University. His research focuses on graph data management and mining.



**Jianliang Xu** (Fellow, IEEE) received the BEng degree in computer science and engineering from Zhejiang University, China, and the PhD degree in computer science from the Hong Kong University of Science and Technology. He is currently a chair professor and the head of the Department of Computer Science, Hong Kong Baptist University. With an h-index of 65, he has authored or coauthored extensively in top-tier venues. His research interests include databases, blockchain, and privacy-aware data management.